

BÀI GIẢNG LẬP TRÌNH MÃ NGUỒN MỞ

GV. Vũ Phú Lộc

MỤC LỤC

Chương 1. Tổng quan về lập trình web với PHP và môi trường phát triển hiện đại	3
1.1 Tổng quan về ngôn ngữ lập trình PHP	3
1.1.1 Lịch sử và vai trò của PHP trong lập trình web	3
1.1.2 Cấu trúc file PHP và cú pháp cơ bản	3
1.1.3 Biến và hằng số trong PHP	4
1.1.4 Toán tử trong PHP	4
1.1.5 Một số hàm cơ bản để xuất dữ liệu	5
1.2 Môi trường phát triển cục bộ với Laragon và VS Code	6
1.2.1 Giới thiệu về Laragon	6
1.2.2 Cài đặt và cấu hình Laragon	6
1.2.3 Giới thiệu Visual Studio Code (VS Code)	7
1.2.4 Tạo và chạy ứng dụng PHP đầu tiên với Laragon	7
1.2.5 Một số lỗi thường gặp và cách khắc phục	8
1.3 Các kiểu dữ liệu cơ bản và cấu trúc điều khiển	8
1.3.1 Kiểu dữ liệu cơ bản trong PHP	8
1.3.2 Cấu trúc điều khiển	9
1.3.3 Cấu trúc lặp (loop)	10
1.3.4 Câu lệnh điều kiện ngắn (Toán tử 3 ngôi)	11
1.4 Hàm và phương thức trong PHP	11
1.4.1 Định nghĩa và gọi hàm trong PHP	11
1.4.2 Tham số mặc định và giá trị trả về	12
1.4.3 Hàm có sẵn trong PHP	13
1.4.4 Kiểm tra và gọi hàm một cách an toàn	14
1.5 Xử lý tập tin và Form	14
1.5.1 Form HTML cơ bản	14
1.5.2 Xử lý form bằng PHP	15
1.5.3 Upload tập tin	16
1.5.4 Đọc và ghi tập tin	17
1.6 Quản lý trạng thái ứng dụng: Session và Cookie	18
1.6.1 Khái niệm về trạng thái trong ứng dụng web	18
1.6.2 Session	18

1.6.3 Cookie	19
1.6.4 So sánh Session và Cookie.....	20
1.6.5 Ứng dụng thực tế.....	20
Chương 2. Lập trình hướng đối tượng (OOP) trong PHP và tích hợp Front-end	21
2.1 Lập trình hướng đối tượng trong PHP	21
2.1.1 Các khái niệm cơ bản trong OOP PHP	21
2.2 Giới thiệu JavaScript và tương tác với PHP	26
2.2.1 Giới thiệu.....	26
2.2.2 JavaScript cơ bản	27
2.2.3 Tương tác JavaScript – PHP	27
2.2.4 JSON – Cầu nối dữ liệu giữa PHP và JavaScript	28
Chương 3. Tương tác cơ sở dữ liệu với MySQL và PDO	30
3.1 Cơ sở dữ liệu MySQL	30
3.1.1 Tổng quan về MySQL.....	30
3.1.2 Thiết kế cơ sở dữ liệu.....	31
3.1.3 Quản trị cơ bản.....	31
3.1.4 Sao lưu và phục hồi dữ liệu.....	32
3.2 Ngôn ngữ truy vấn cấu trúc (SQL)	32
3.2.1 Các câu lệnh cơ bản	32
3.2.2 Truy vấn nâng cao	33
3.3 Lập trình tương tác cơ sở dữ liệu với PDO.....	35
3.3.1 Giới thiệu PDO.....	35
3.3.2 Kết nối CSDL với PDO	36
3.3.3 Thực hiện truy vấn an toàn.....	36
3.3.4 Các thao tác CRUD với PDO.....	39
3.4 Các kỹ thuật hiển thị dữ liệu nâng cao.....	40
3.4.1 Phân trang (Pagination).....	40
3.4.2 Tìm kiếm (Searching)	41

Chương 1. Tổng quan về lập trình web với PHP và môi trường phát triển hiện đại

Chương 1 cung cấp kiến thức nền tảng về ngôn ngữ lập trình PHP – một trong những công nghệ phổ biến trong lập trình web server-side. Sinh viên sẽ tìm hiểu lịch sử, vai trò và cú pháp cơ bản của PHP, cách thiết lập môi trường phát triển hiện đại bằng Laragon và VS Code, cùng với các kiểu dữ liệu, cấu trúc điều khiển, hàm, xử lý form, tập tin và quản lý trạng thái ứng dụng. Đây là bước khởi đầu quan trọng để chuẩn bị cho các chương nâng cao tiếp theo về hướng đối tượng, cơ sở dữ liệu và framework Laravel.

1.1 Tổng quan về ngôn ngữ lập trình PHP

1.1.1 Lịch sử và vai trò của PHP trong lập trình web

PHP (viết tắt của "PHP: Hypertext Preprocessor") là một ngôn ngữ lập trình mã nguồn mở được thiết kế đặc biệt để phát triển các ứng dụng web chạy phía máy chủ (server-side). Ban đầu, PHP được phát triển bởi Rasmus Lerdorf vào năm 1994 dưới dạng một tập hợp các script CGI đơn giản được viết bằng ngôn ngữ C, dùng để theo dõi lượt truy cập trang cá nhân của ông. Về sau, PHP được phát triển thành một ngôn ngữ lập trình hoàn chỉnh và ngày càng được cải tiến mạnh mẽ qua các phiên bản.

Các mốc phát triển chính của PHP:

- 1994: Ra đời dưới tên gọi ban đầu là Personal Home Page Tools (PHP Tools).
- 1995: Phát hành phiên bản đầu tiên PHP/FI.
- 1997: PHP 3 ra đời, hỗ trợ cơ sở dữ liệu và cú pháp lập trình hiện đại hơn.
- 2000: PHP 4 phát hành, sử dụng Zend Engine 1.0.
- 2004: PHP 5 hỗ trợ mạnh lập trình hướng đối tượng (OOP) với Zend Engine 2.
- 2015: PHP 7 cải thiện hiệu năng vượt bậc, bỏ qua phiên bản 6.
- 2020 trở đi: PHP 8+ bổ sung nhiều tính năng hiện đại như JIT compiler, attributes, union types...

Vai trò của PHP trong lập trình web:

- Là ngôn ngữ server-side phổ biến nhất để xây dựng các website động.
- Dễ học, dễ triển khai và hỗ trợ nhiều hệ điều hành và máy chủ web.
- Được sử dụng để phát triển nhiều hệ thống quản lý nội dung lớn như WordPress, Joomla, Drupal.
- Là nền tảng của nhiều framework hiện đại như Laravel, Symfony, CodeIgniter.

PHP đã và đang là công cụ quan trọng trong việc phát triển các ứng dụng web từ đơn giản đến phức tạp nhờ tính linh hoạt, hiệu suất cao và cộng đồng phát triển mạnh mẽ.

1.1.2 Cấu trúc file PHP và cú pháp cơ bản

Một chương trình PHP thường được lưu trong file có phần mở rộng là .php. Trong file đó, các đoạn mã PHP được đặt trong cặp thẻ mở và đóng:

```
<?php
    // Mã PHP ở đây
?>
```

Bên ngoài cặp thẻ này, bạn có thể viết mã HTML như bình thường. Đây là đặc điểm nổi bật của PHP: *có thể nhúng vào trong mã HTML một cách linh hoạt.*

Ví dụ đơn giản:

```
<!DOCTYPE html>
<html>
<head>
    <title>Chào mừng</title>
</head>
<body>
    <h1><?php echo "Xin chào, sinh viên!"; ?></h1>
</body>
</html>
```

Kết quả: Trình duyệt sẽ hiển thị dòng tiêu đề "Xin chào, sinh viên!" được sinh ra bởi PHP.

1.1.3 Biến và hằng số trong PHP

Biến

- Trong PHP, tên biến bắt đầu bằng ký tự \$ và phân biệt chữ hoa – chữ thường.
- Biến không cần khai báo kiểu dữ liệu, PHP tự động xác định kiểu dựa vào giá trị gán.

```
<?php
$name = "Nguyễn Văn A";
$age = 20;
$score = 8.5;
?>
```

Một số quy tắc đặt tên biến:

- Bắt đầu bằng ký tự \$, theo sau là chữ cái hoặc gạch dưới.
- Không bắt đầu bằng số.
- Không có khoảng trắng, không chứa ký tự đặc biệt (ngoại trừ _).
- Ví dụ tên biến hợp lệ: \$hoTen, \$diemToan, \$so_luong.

Hằng số

Hằng số được khai báo bằng hàm `define()` và không thể thay đổi giá trị sau khi đã được định nghĩa.

Ví dụ:

```
<?php
define("PI", 3.14159);
echo PI; // Kết quả: 3.14159
?>
```

1.1.4 Toán tử trong PHP

PHP hỗ trợ nhiều loại toán tử:

a. Toán tử số học

Toán tử	Mô tả	Ví dụ
+	Cộng	$\$a + \b
-	Trừ	$\$a - \b
*	Nhân	$\$a * \b
/	Chia	$\$a / \b
%	Chia lấy dư	$\$a \% \b

b. Toán tử gán

Toán tử	Mô tả	Ví dụ
=	Gán	$\$a = 5;$
+=	Cộng rồi gán	$\$a += 3;$
-=	Trừ rồi gán	$\$a -= 2;$

c. Toán tử so sánh

Toán tử	Mô tả	Ví dụ
==	So sánh bằng giá trị	$\$a == \b
===	So sánh bằng cả kiểu	$\$a === \b
!=, <>	Khác	$\$a != \b
>, <, >=, <=	So sánh lớn hơn, nhỏ hơn	$\$a > \b

d. Toán tử logic

Toán tử	Mô tả	Ví dụ
&& hoặc and	Và	$\$a > 0 \ \&\& \ \$b < 10$
hoặc or	Hoặc	$\$a == 1 \ \ \$a == 2$
!	Phủ định	$! (\$a > 0)$

1.1.5 Một số hàm cơ bản để xuất dữ liệu

- `echo`: Xuất nội dung ra trình duyệt (không trả giá trị).
- `print`: Giống `echo`, nhưng trả về giá trị 1.
- `var_dump()`: Hiển thị kiểu và giá trị của biến (rất hữu ích khi debug).

Ví dụ:

```
<?php
$ten = "Lập trình PHP";
$so = 10;
echo $ten;
print "<br>";
var_dump($so);
?>
```

Kết quả:

```
Lập trình PHP
int(10)
```

1.2 Môi trường phát triển cục bộ với Laragon và VS Code

1.2.1 Giới thiệu về Laragon

Laragon là một phần mềm giúp thiết lập môi trường phát triển web cục bộ (local development environment) một cách nhanh chóng và thuận tiện. Nó hỗ trợ PHP, MySQL, Apache/Nginx, Node.js, Redis... và có khả năng hoạt động “di động” (portable), nghĩa là bạn có thể sao chép thư mục Laragon sang máy khác mà không cần cài đặt lại.

Ưu điểm nổi bật của Laragon:

- Cài đặt dễ dàng, giao diện thân thiện.
 - Tích hợp sẵn Apache/Nginx, MySQL/MariaDB, PHP, phpMyAdmin.
 - Hỗ trợ tự động tạo Virtual Host và domain ảo (VD: myapp.test).
 - Tích hợp Terminal, Composer, Node.js và nhiều công cụ khác.
 - Tốc độ khởi động nhanh, phù hợp cho máy cấu hình trung bình.
- So với XAMPP, Laragon nhẹ hơn, chạy mượt hơn và quản lý dự án linh hoạt hơn.

1.2.2 Cài đặt và cấu hình Laragon

Các bước cài đặt Laragon:

Tải Laragon:

- Truy cập trang chủ: <https://laragon.org>
- Chọn bản "Laragon Full" (có sẵn Apache, MySQL, PHP, Node.js...).

Cài đặt:

- Chạy file .exe, chọn thư mục cài đặt (VD: C:\Laragon)
- Trong quá trình cài đặt, có thể chọn cấu hình mặc định.

Khởi động dịch vụ:

- Mở Laragon → Nhấn nút Start All để khởi động Apache và MySQL.
- Kiểm tra bằng cách mở trình duyệt → truy cập: <http://localhost>

Tạo Project PHP mới:

- Nhấn chuột phải vào biểu tượng Laragon (system tray) → Menu Quick App > Blank → Đặt tên dự án (VD: myproject)
- Laragon sẽ tạo folder C:\Laragon\www\myproject
- Tự động tạo domain ảo http://myproject.test

Lưu ý: Nếu domain ảo chưa hoạt động, hãy kiểm tra file hosts và bật tùy chọn “Auto Virtual Hosts”.

1.2.3 Giới thiệu Visual Studio Code (VS Code)

Visual Studio Code (VS Code) là trình soạn thảo mã nguồn miễn phí, nhẹ, mạnh mẽ và mở rộng linh hoạt, được phát triển bởi Microsoft.

Tính năng nổi bật của VS Code:

- Hỗ trợ nhiều ngôn ngữ: PHP, HTML, CSS, JS, Python, C#...
- Tự động gợi ý cú pháp, tô màu mã nguồn.
- Gỡ lỗi (debug), tích hợp terminal, Git.
- Cài đặt extension để hỗ trợ framework hoặc ngôn ngữ cụ thể.

Cài đặt VS Code:

- Tải tại: <https://code.visualstudio.com>
- Cài đặt theo mặc định.
- Mở thư mục dự án PHP (VD: myproject) bằng File > Open Folder

Extension cần thiết cho lập trình PHP:

Extension	Mô tả
PHP Intelephense	Gợi ý, kiểm tra lỗi, phân tích mã PHP
PHP Server	Chạy nhanh file PHP trong trình duyệt
Live Server	Tự động refresh khi lưu file HTML/CSS
Laravel Blade Snippets	Hỗ trợ syntax Blade của Laravel
Prettier	Định dạng mã tự động
GitLens	Hỗ trợ Git trong VS Code

Lưu ý: Cài extension bằng cách bấm biểu tượng Extensions (Ctrl+Shift+X) rồi tìm tên.

1.2.4 Tạo và chạy ứng dụng PHP đầu tiên với Laragon

Sau khi cài Laragon và VS Code, chúng ta sẽ thực hiện một ứng dụng PHP đơn giản để kiểm tra hoạt động môi trường.

Bước 1: Tạo dự án

- Vào Laragon > Menu Quick App > Blank > Đặt tên là hello-php.

- Laragon sẽ tạo thư mục C:\Laragon\www\hello-php

Bước 2: Viết mã PHP đầu tiên

- Mở VS Code → Open Folder C:\Laragon\www\hello-php
- Tạo file index.php với nội dung:

```
<?php
    $name = "Sinh viên!";
    echo "<h1>Xin chào, $name</h1>";
?>
```

Bước 3: Mở trình duyệt để kiểm tra

- Truy cập địa chỉ: http://hello-php.test hoặc http://localhost/hello-php
- Kết quả sẽ hiển thị: "Xin chào, Sinh viên!"

1.2.5 Một số lỗi thường gặp và cách khắc phục

Vấn đề	Nguyên nhân	Cách xử lý
Trình duyệt báo lỗi 404	Sai đường dẫn project	Kiểm tra URL, folder www
Không chạy domain .test	Hosts chưa cập nhật	Mở Laragon > Tools > Auto Hosts
Dịch vụ Apache/MySQL không chạy	Port bị chiếm	Đổi port hoặc tắt phần mềm xung đột
Không gợi ý mã trong VS Code	Thiếu extension PHP Intelephense	Cài và reload extension

1.3 Các kiểu dữ liệu cơ bản và cấu trúc điều khiển

Trong lập trình PHP, hiểu và sử dụng đúng kiểu dữ liệu và cấu trúc điều khiển là nền tảng để xử lý logic trong các ứng dụng web. Mục này giúp sinh viên nắm được cách khai báo, sử dụng và vận dụng các thành phần cốt lõi của PHP khi viết mã.

1.3.1 Kiểu dữ liệu cơ bản trong PHP

PHP là ngôn ngữ **dynamically typed** – nghĩa là không cần khai báo kiểu dữ liệu rõ ràng khi tạo biến. PHP sẽ tự xác định kiểu dựa vào giá trị được gán cho biến.

Các kiểu dữ liệu chính:

Loại dữ liệu	Ví dụ giá trị	Mô tả ngắn
Integer	10, -7, 2025	Số nguyên
Float/Double	3.14, -0.5, 8.75	Số thực (số có phần thập phân)
String	"PHP", 'Xin chào'	Chuỗi ký tự, có thể dùng nháy đơn hoặc nháy kép

Loại dữ liệu	Ví dụ giá trị	Mô tả ngắn
Boolean	true, false	Biểu diễn đúng/sai (dùng cho điều kiện logic)
Array	[1, 2, 3], ["a"=>"b"]	Tập hợp giá trị
Object	new ClassName()	Đối tượng (sẽ học chi tiết ở chương OOP)
NULL	null	Biến không có giá trị hoặc bị xóa

Ghi nhớ: Sử dụng `var_dump($bien)` để kiểm tra kiểu dữ liệu và giá trị của biến.

Ví dụ:

```
<?php
$ten = "PHP";
$tuoi = 20;
$diem = 8.5;
$laSinhVien = true;

var_dump($ten);           // string(3) "PHP"
var_dump($tuoi);          // int(20)
var_dump($diem);          // float(8.5)
var_dump($laSinhVien);    // bool(true)
?>
```

1.3.2 Cấu trúc điều khiển

Các cấu trúc điều khiển giúp chương trình PHP ra quyết định hoặc lặp lại hành động tùy theo điều kiện. Đây là công cụ quan trọng để tạo nên logic xử lý động trong ứng dụng.

a. Cấu trúc rẽ nhánh (if – else – elseif)

Cú pháp:

```
if (điều_kiện) {
    // Nếu đúng thì thực hiện đoạn này
} elseif (điều_kiện_khác) {
    // Nếu điều kiện khác đúng thì thực hiện
} else {
    // Nếu không điều kiện nào đúng thì thực hiện phần này
}
```

Ví dụ:

```
<?php
$diem = 7.5;

if ($diem >= 9) {
    echo "Xuất sắc";
} elseif ($diem >= 7) {
    echo "Khá";
}
```

```
} else {  
    echo "Trung bình hoặc yếu";  
}  
?>
```

b. Cấu trúc switch – case

Dùng khi cần so sánh một biến với nhiều giá trị cụ thể.

Cú pháp:

```
switch ($bien) {  
    case 'giatri1':  
        // hành động nếu đúng  
        break;  
    case 'giatri2':  
        // hành động khác  
        break;  
    default:  
        // nếu không khớp giá trị nào  
}
```

Ví dụ:

```
<?php  
$ngay = 3;  
  
switch ($ngay) {  
    case 1:  
        echo "Chủ nhật";  
        break;  
    case 2:  
        echo "Thứ hai";  
        break;  
    default:  
        echo "Ngày không hợp lệ";  
}  
?>
```

1.3.3 Cấu trúc lặp (loop)

a. Vòng lặp for

Lặp với số lần xác định trước.

```
for ($i = 1; $i <= 5; $i++) {  
    echo "Lần thứ $i <br>";  
}
```

b. Vòng lặp while

Lặp khi điều kiện đúng.

```
$i = 1;
```

```
while ($i <= 3) {
    echo "Số $i <br>";
    $i++;
}
```

c. Vòng lặp do...while

Thực hiện ít nhất một lần rồi mới kiểm tra điều kiện.

```
$i = 5;
do {
    echo "Giá trị: $i <br>";
    $i--;
} while ($i > 0);
```

d. Vòng lặp foreachs

Chuyên dùng để lặp qua mảng.

```
$dsMonHoc = ["PHP", "JavaScript", "HTML"];

foreach ($dsMonHoc as $mon) {
    echo "Môn: $mon <br>";
}
```

1.3.4 Câu lệnh điều kiện ngắn (Toán tử 3 ngôi)

Cú pháp:

```
$result = (điều_kiện) ? giá_trị_đúng : giá_trị_sai;
```

Ví dụ:

```
$diem = 8;
$ketQua = ($diem >= 5) ? "Đậu" : "Rớt";
echo $ketQua;
```

1.4 Hàm và phương thức trong PHP

Trong lập trình, **hàm (function)** là khối mã thực hiện một nhiệm vụ cụ thể và có thể được gọi lại nhiều lần. Việc sử dụng hàm giúp mã dễ đọc, dễ bảo trì và tái sử dụng hiệu quả hơn. PHP cung cấp khả năng định nghĩa hàm riêng và cũng có sẵn rất nhiều hàm dựng sẵn (**built-in functions**) để xử lý chuỗi, mảng, thời gian, tập tin, v.v.

Trong phạm vi chương này, phương thức (method) được hiểu đơn giản là hàm nằm trong lớp – phần này sẽ được học kỹ hơn ở chương Lập trình hướng đối tượng.

1.4.1 Định nghĩa và gọi hàm trong PHP

Cú pháp định nghĩa hàm:

```
function tenHam([danh_sach_tham_so]) {
    // Khối lệnh thực thi
    return [gia_tri_tra_ve];
}
```

- Tên hàm: Bắt đầu bằng chữ cái hoặc dấu gạch dưới, không được trùng tên với hàm đã có.

- Tham số (có thể có hoặc không): Cho phép truyền dữ liệu vào hàm.
- Lệnh `return` (tùy chọn): Trả kết quả từ hàm về nơi gọi.

Ví dụ 1: Hàm không tham số và không trả giá trị

```
<?php
function chaoMung() {
    echo "Xin chào sinh viên PHP!";
}

chaoMung(); // Gọi hàm
?>
```

Ví dụ 2: Hàm có tham số

```
<?php
function tinhTong($a, $b) {
    $tong = $a + $b;
    return $tong;
}

$ktq = tinhTong(5, 7);
echo "Tổng là: $ktq"; // Kết quả: Tổng là: 12
?>
```

Có thể gọi hàm nhiều lần với giá trị khác nhau để tái sử dụng logic.

1.4.2 Tham số mặc định và giá trị trả về

PHP cho phép gán giá trị mặc định cho tham số trong định nghĩa hàm.

Ví dụ:

```
function chao($ten = "bạn") {
    echo "Xin chào, $ten <br>";
}

chao("Nam"); // In: Xin chào, Nam
chao(); // In: Xin chào, bạn
```

Trả về giá trị (`return`)

- Một hàm có thể trả về kết quả để xử lý tiếp.
- Nếu không có `return`, mặc định trả về `null`.

```
function binhPhuong($x) {
    return $x * $x;
}

$sq = binhPhuong(6); // 36
```

1.4.3 Hàm có sẵn trong PHP

PHP cung cấp hàng trăm hàm dựng sẵn để xử lý chuỗi, mảng, số, thời gian, v.v. Dưới đây là một số nhóm hàm thông dụng.

a. Nhóm hàm xử lý chuỗi (string)

Hàm	Mô tả
<code>strlen(\$str)</code>	Trả độ dài chuỗi
<code>strtolower()</code>	Chuyển thành chữ thường
<code>strtoupper()</code>	Chuyển thành chữ hoa
<code>substr(\$str, \$a, \$b)</code>	Trích xuất chuỗi con
<code>strpos(\$str, \$find)</code>	Tìm vị trí chuỗi con trong chuỗi gốc

Ví dụ:

```
$str = "Lập trình PHP";  
echo strlen($str); // 13  
echo strtoupper($str); // LẬP TRÌNH PHP
```

b. Nhóm hàm xử lý mảng (array)

Hàm	Mô tả
<code>count(\$arr)</code>	Đếm số phần tử trong mảng
<code>in_array(\$x, \$arr)</code>	Kiểm tra phần tử có tồn tại trong mảng không
<code>array_push(\$arr, \$val)</code>	Thêm phần tử vào cuối mảng
<code>array_merge(\$a, \$b)</code>	Trộn hai mảng
<code>sort(\$arr)</code>	Sắp xếp mảng tăng dần

Ví dụ:

```
$mon = ["PHP", "JS"];  
array_push($mon, "HTML");  
print_r($mon); // ["PHP", "JS", "HTML"]
```

c. Nhóm hàm xử lý thời gian (Date/Time)

Hàm	Mô tả
<code>date(\$format)</code>	Lấy ngày giờ hiện tại theo định dạng
<code>time()</code>	Lấy timestamp hiện tại
<code>strtotime(\$str)</code>	Chuyển chuỗi ngày về timestamp

Ví dụ:

```
echo date("d/m/Y H:i:s"); // VD: 07/08/2025 09:15:23
```

Có thể dùng `date("l")` để hiển thị thứ trong tuần: Monday, Tuesday...

1.4.4 Kiểm tra và gọi hàm một cách an toàn

PHP có hàm `function_exists("ten_ham")` để kiểm tra hàm đã tồn tại hay chưa trước khi gọi.

```
if (function_exists("tinhTong")) {  
    echo tinhTong(3, 4);  
}
```

Việc này đặc biệt hữu ích khi làm việc với thư viện mở rộng hoặc import nhiều file PHP.

1.5 Xử lý tập tin và Form

Biểu mẫu (form) là phương tiện giao tiếp chính giữa người dùng và ứng dụng web. Trong PHP, biểu mẫu thường được sử dụng để thu thập dữ liệu người dùng như thông tin đăng nhập, tìm kiếm, đăng ký, phản hồi,... Sau khi người dùng nhập dữ liệu và nhấn nút gửi (submit), dữ liệu sẽ được gửi đến server để xử lý bằng mã PHP.

1.5.1 Form HTML cơ bản

Một biểu mẫu HTML thường gồm:

- Thẻ `<form>`: Khai báo biểu mẫu.
- Thuộc tính `action`: Chỉ định URL nơi dữ liệu sẽ được gửi đến.
- Thuộc tính `method`: Phương thức gửi dữ liệu (GET hoặc POST).
- Các điều khiển nhập liệu như: `<input>`, `<textarea>`, `<select>`, `<button>`...

Ví dụ:

```
<form action="xuly.php" method="post">  
    Họ tên: <input type="text" name="hoten"><br>  
    Email: <input type="email" name="email"><br>  
    <input type="submit" value="Gửi">  
</form>
```

Phân biệt phương thức GET và POST

Tiêu chí	GET	POST
Hiển thị URL	Tham số hiển thị trên thanh địa chỉ	Không hiển thị trên URL
Bảo mật	Kém bảo mật	Bảo mật hơn
Dung lượng	Giới hạn (thường 2048 ký tự)	Gần như không giới hạn
Sử dụng khi	Truy vấn, tìm kiếm dữ liệu	Gửi dữ liệu nhạy cảm (form đăng ký)

1.5.2 Xử lý form bằng PHP

1.5.2.1 Truy xuất dữ liệu biểu mẫu bằng PHP

PHP cung cấp các biến toàn cục:

- \$_GET: Chứa dữ liệu gửi từ form bằng phương thức GET
- \$_POST: Chứa dữ liệu gửi từ form bằng phương thức POST
- \$_REQUEST: Chứa cả GET lẫn POST.

Ví dụ 1: Sử dụng \$_POST

HTML:

```
<form method="post" action="xuly.php">
    Nhập tên: <input type="text" name="ten">
    <input type="submit" value="Gửi">
</form>
```

xuly.php:

```
<?php
$ten = $_POST['ten'];
echo "Xin chào, $ten!";
?>
```

1.5.2.2 Kiểm tra dữ liệu tồn tại

Trước khi xử lý, nên kiểm tra xem dữ liệu có được gửi không bằng hàm `isset()` hoặc `!empty()`.

Ví dụ:

```
<?php
if (isset($_POST['ten'])) {
    $ten = $_POST['ten'];
    echo "Xin chào, $ten!";
} else {
    echo "Vui lòng nhập tên.";
}
?>
```

1.5.2.3 Bảo vệ dữ liệu đầu vào

Việc xử lý dữ liệu từ người dùng cần có biện pháp bảo vệ để tránh lỗi hoặc tấn công:

- Trim dữ liệu: `trim($ten)`
- Loại bỏ mã HTML: `strip_tags($ten)`
- Chuyển ký tự đặc biệt: `htmlspecialchars($ten)`
- Kiểm tra dữ liệu rỗng, định dạng...

Ví dụ:

```
<?php
```



```

if (isset($_POST['email'])) {
    $email = trim($_POST['email']);
    $email = htmlspecialchars($email);
    echo "Email của bạn là: $email";
}
?>

```

1.5.2.4 Biểu mẫu gửi đến chính nó (self-submitting form)

Một cách phổ biến để xử lý biểu mẫu là gửi đến chính trang hiện tại (action = "").

Ví dụ:

```

<form method="post" action="">
    Nhập tuổi: <input type="number" name="tuoi">
    <input type="submit" value="Kiểm tra">
</form>

<?php
if (isset($_POST['tuoi'])) {
    $tuoi = $_POST['tuoi'];
    if ($tuoi >= 18) {
        echo "Bạn đã đủ tuổi.";
    } else {
        echo "Bạn chưa đủ tuổi.";
    }
}
}
?>

```

1.5.2.5 Một số lỗi thường gặp

Lỗi	Nguyên nhân
Undefined index	Truy cập biến chưa tồn tại trong \$_POST hoặc \$_GET
Dữ liệu không hiển thị	Quên echo, lỗi cú pháp hoặc action không chính xác
Gửi sai định dạng dữ liệu	Không kiểm tra đầu vào, thiếu xử lý định dạng dữ liệu

1.5.3 Upload tập tin

Để upload tập tin, form phải có thuộc tính enctype="multipart/form-data":

```

<form method="post" enctype="multipart/form-data" action="upload.php">
    <input type="file" name="myfile"><br>
    <input type="submit" value="Upload">
</form>

```

Tập *upload.php* sẽ xử lý:

```

<?php
if (isset($_FILES['myfile'])) {
    $file = $_FILES['myfile'];
}

```

```

        if ($file['error'] == 0) {
            move_uploaded_file($file['tmp_name'], "uploads/" .
$file['name']);
            echo "Upload thành công!";
        } else {
            echo "Lỗi upload: " . $file['error'];
        }
    }
}
?>

```

Các thông tin trong \$_FILES:

- \$_FILES['myfile']['name']: Tên tập tin.
- \$_FILES['myfile']['tmp_name']: Đường dẫn tạm trên server.
- \$_FILES['myfile']['error']: Mã lỗi.
- \$_FILES['myfile']['size']: Dung lượng tập tin.

1.5.4 Đọc và ghi tập tin

PHP cung cấp các hàm để thao tác với tập tin văn bản.

a. Ghi nội dung vào tập tin

```

<?php
$fp = fopen("data.txt", "w");
fwrite($fp, "Dòng 1\n");
fwrite($fp, "Dòng 2\n");
fclose($fp);
?>

```

- "w": Ghi mới, xóa nội dung cũ nếu tồn tại.

b. Đọc nội dung từ tập tin

Đọc từng dòng từ file

```

<?php
$fp = fopen("data.txt", "r");

while (!feof($fp)) {
    $line = fgets($fp);
    echo $line . "<br>";
}

fclose($fp);
?>

```

- fgets(): Đọc từng dòng.
- feof(): Kiểm tra cuối tập tin.

Đọc toàn bộ file

```
<?php
$fn = "data.txt";
$fp = fopen($fn, "r");

$cnt = fread($fp, filesize($fn));

echo $cnt;
fclose($fp);
?>
```

1.6 Quản lý trạng thái ứng dụng: Session và Cookie

1.6.1 Khái niệm về trạng thái trong ứng dụng web

- *HTTP là giao thức không lưu trạng thái (stateless)*: Mỗi request gửi lên server được xử lý độc lập, không tự động ghi nhớ các request trước.
- Do đó, nếu cần duy trì thông tin người dùng (ví dụ: đã đăng nhập, giỏ hàng...), lập trình viên cần cơ chế quản lý trạng thái.

Có hai cách phổ biến:

- **Session** – Lưu trữ dữ liệu trên server.
- **Cookie** – Lưu trữ dữ liệu trên trình duyệt người dùng.

1.6.2 Session

Khái niệm

- Session là vùng lưu trữ dữ liệu trên server cho từng người dùng riêng biệt.
- Mỗi session được định danh bằng một Session ID (thường lưu trong cookie hoặc truyền qua URL).

Cách sử dụng trong PHP

a. Khởi tạo session

```
<?php
session_start(); // Bắt buộc phải gọi trước khi xuất HTML

$_SESSION['username'] = "nguyenvana";
$_SESSION['role'] = "admin";

echo "Session đã được tạo!";
?>
```

b. Truy xuất dữ liệu từ session

```
<?php
session_start();
echo "Xin chào " . $_SESSION['username'];
echo "Quyền hạn: " . $_SESSION['role'];
?>
```

c. Xóa session

```
<?php
session_start();
session_unset(); // Xóa toàn bộ biến session
session_destroy(); // Hủy session
?>
```

Lưu ý:

- Session tồn tại cho đến khi người dùng đóng trình duyệt hoặc đến khi hết thời gian timeout trên server.
- An toàn hơn Cookie vì dữ liệu lưu ở server.

1.6.3 Cookie

Khái niệm

- Cookie là một tệp nhỏ lưu trên máy người dùng, chứa thông tin mà server muốn “nhớ” ở phía client.
- Thường dùng để lưu thông tin tùy chỉnh giao diện, token đăng nhập, giỏ hàng nhỏ, v.v.

Cách sử dụng trong PHP

a. Tạo cookie

```
<?php
setcookie("username", "nguyenvana", time() + 3600, "/");
// time() + 3600: hết hạn sau 1 giờ
?>
```

b. Đọc cookie

```
<?php
if (isset($_COOKIE['username'])) {
    echo "Xin chào " . $_COOKIE['username'];
} else {
    echo "Cookie chưa tồn tại.";
}
?>
```

c. Xóa cookie

```
<?php
setcookie("username", "", time() - 3600, "/"); // Đặt thời gian hết hạn về quá khứ
?>
```

Lưu ý:

- Cookie có thể bị người dùng chỉnh sửa, vì vậy không nên lưu dữ liệu nhạy cảm.
- Giới hạn dung lượng (~4KB mỗi cookie) và số lượng cookie.

1.6.4 So sánh Session và Cookie

Đặc điểm	Session	Cookie
Nơi lưu trữ	Server	Trình duyệt người dùng
Bảo mật	Cao hơn	Thấp hơn
Dung lượng	Lớn (tùy server)	~4KB
Thời gian tồn tại	Khi trình duyệt đóng hoặc timeout	Theo thời gian hết hạn do lập trình viên đặt
Tốc độ truy xuất	Nhanh hơn	Phụ thuộc tốc độ đọc/ghi trình duyệt

1.6.5 Ứng dụng thực tế

- **Session:** Giữ trạng thái đăng nhập, giỏ hàng thương mại điện tử.
- **Cookie:** Lưu tùy chọn ngôn ngữ, ghi nhớ tên đăng nhập.

Chương 2. Lập trình hướng đối tượng (OOP) trong PHP và tích hợp Front-end

Chương 2 sẽ giúp sinh viên nắm vững các khái niệm và kỹ thuật lập trình hướng đối tượng (OOP) trong PHP, từ các thành phần cơ bản như lớp, đối tượng, thuộc tính, phương thức, đến các đặc trưng quan trọng của OOP như tính đóng gói, kế thừa, đa hình. Bên cạnh đó, chương cũng giới thiệu cách tích hợp Front-end thông qua JavaScript, DOM manipulation và AJAX để tạo ra các ứng dụng web tương tác mạnh mẽ hơn. Sau khi học xong chương này, sinh viên có thể thiết kế các ứng dụng PHP có cấu trúc tốt, dễ bảo trì và mở rộng, đồng thời nâng cao trải nghiệm người dùng thông qua các kỹ thuật lập trình front-end.

2.1 Lập trình hướng đối tượng trong PHP

Lập trình hướng đối tượng (Object-Oriented Programming – OOP) là cách tổ chức mã nguồn dựa trên đối tượng thay vì chỉ tập trung vào các hàm rời rạc. Mỗi đối tượng được mô tả thông qua **lớp (class)**, chứa **thuộc tính (property)** mô tả trạng thái và **phương thức (method)** mô tả hành vi.

Từ PHP 5 trở đi, PHP hỗ trợ OOP đầy đủ như các ngôn ngữ hiện đại, bao gồm đóng gói, kế thừa, đa hình, lớp trừu tượng, giao diện, và tự động nạp lớp.

Ưu điểm của OOP:

- Mã nguồn rõ ràng, dễ quản lý.
- Dễ mở rộng và bảo trì.
- Cho phép tái sử dụng mã qua kế thừa và thành phần.

2.1.1 Các khái niệm cơ bản trong OOP PHP

2.1.1.1 Lớp (Class)

- Định nghĩa: Lớp là một "bản thiết kế" mô tả cấu trúc và hành vi của đối tượng.
- Cú pháp:

```
class TenLop {  
    // Thuộc tính  
    public $thuocTinh;  
  
    // Phương thức  
    public function phuongThuc() {  
        echo "Hành động của đối tượng";  
    }  
}
```

Ghi chú:

- Tên lớp nên viết theo PascalCase.
- Mỗi lớp có thể chứa nhiều thuộc tính và phương thức.
- Không có dấu \$ trước tên lớp.

2.1.1.2 Đối tượng (Object)

- Định nghĩa: Đối tượng là một thể hiện (instance) cụ thể của lớp.

- Tạo đối tượng:

```
$obj = new TenLop();
$obj->thuocTinh = "Giá trị";
$obj->phuongThuc();
```

Ghi chú:

- Dùng toán tử -> để truy cập thuộc tính hoặc phương thức.
- Mỗi đối tượng có vùng nhớ riêng cho thuộc tính của nó.

2.1.1.3 Thuộc tính (Property)

- Định nghĩa: Biến được khai báo trong lớp, lưu trữ trạng thái của đối tượng.
- Phạm vi truy cập (Access Modifiers):
 - public: truy cập từ mọi nơi.
 - private: chỉ truy cập bên trong lớp khai báo.
 - protected: truy cập trong lớp khai báo và lớp kế thừa.
- Ví dụ:

```
class User {
    public $name;        // công khai
    private $password;   // riêng tư

    public function setPassword($pwd) {
        $this->password = $pwd;
    }
}
```

2.1.1.4 Phương thức (Method)

- Định nghĩa: Hàm được khai báo trong lớp, định nghĩa hành vi của đối tượng.
- Ví dụ:

```
class Car {
    public $brand;

    public function drive() {
        echo $this->brand . " is driving...";
    }
}

$c = new Car();
$c->brand = "Toyota";
$c->drive(); // Toyota is driving...
```

Ghi chú:

- \$this là biến đặc biệt trỏ đến chính đối tượng hiện tại.

- Phương thức có thể nhận tham số và trả về giá trị như hàm bình thường.

2.1.1.5 Constructor và Destructor

- Constructor: Hàm đặc biệt `__construct()` được gọi ngay khi tạo đối tượng.
- Destructor: Hàm đặc biệt `__destruct()` được gọi khi đối tượng bị hủy.
- Ví dụ:

```
class Product {
    public $name;

    public function __construct($n) {
        $this->name = $n;
        echo "Sản phẩm {$n} được tạo.<br>";
    }

    public function __destruct() {
        echo "Sản phẩm {$this->name} bị hủy.";
    }
}

$p = new Product("Laptop");
```

2.1.1.6 Kế thừa (Inheritance)

- Định nghĩa: Lớp con kế thừa thuộc tính và phương thức từ lớp cha.
- Cú pháp: `class LopCon extends LopCha`
- Ví dụ:

```
class Animal {
    public function speak() {
        echo "Animal sound";
    }
}

class Dog extends Animal {
    public function speak() {
        echo "Woof!";
    }
}

$d = new Dog();
$d->speak(); // Woof!
```

Ghi chú:

- Lớp con có thể ghi đè (override) phương thức của lớp cha.
- Sử dụng `parent::tenPhuongThuc()` để gọi phương thức lớp cha.

2.1.1.7 Đa hình (Polymorphism)

- Khái niệm: Cùng một phương thức nhưng hành vi khác nhau tùy đối tượng.
- Ví dụ:

```
$animals = [new Animal(), new Dog()];  
foreach ($animals as $a) {  
    $a->speak(); // Animal sound / Woof!  
}
```

2.1.1.8 Đóng gói (Encapsulation)

- Khái niệm: Giấu thông tin bên trong lớp, chỉ cho phép truy cập qua phương thức công khai.
- Ví dụ:

```
class BankAccount {  
    private $balance = 0;  
  
    public function deposit($amount) {  
        if ($amount > 0) {  
            $this->balance += $amount;  
        }  
    }  
  
    public function getBalance() {  
        return $this->balance;  
    }  
}
```

2.1.1.9 Abstract Class (Lớp trừu tượng)

- Định nghĩa: Lớp trừu tượng là lớp không thể tạo đối tượng trực tiếp, chỉ dùng để làm lớp cha cho các lớp con. Nó có thể chứa:
 - Phương thức bình thường (có nội dung).
 - Phương thức trừu tượng (khai báo nhưng chưa có nội dung, bắt buộc lớp con phải định nghĩa lại).
- Cú pháp:

```
abstract class Animal {  
    abstract public function makeSound(); // phương thức trừu tượng  
  
    public function sleep() {  
        echo "Sleeping...";  
    }  
}  
  
class Cat extends Animal {  
    public function makeSound() {  
        echo "Meow";  
    }  
}
```

```

    }
}

$c = new Cat();
$c->makeSound(); // Meow

```

Ghi chú:

- Nếu lớp có ít nhất 1 phương thức trừu tượng, bắt buộc phải khai báo là abstract.
- Các lớp con phải triển khai (implement) toàn bộ phương thức trừu tượng.

2.1.1.10 Interface

- Định nghĩa: Interface giống như một bản cam kết, chỉ chứa khai báo phương thức, không có nội dung. Lớp muốn sử dụng phải triển khai (implements) tất cả phương thức trong interface.
- Cú pháp:

```

interface Logger {
    public function log($message);
}

class FileLogger implements Logger {
    public function log($message) {
        echo "Ghi log vào file: $message";
    }
}

$logger = new FileLogger();
$logger->log("Hello");

```

Khác biệt giữa *Abstract Class* và *Interface*:

Abstract Class	Interface
Có thể chứa thuộc tính và phương thức có nội dung	Chỉ chứa khai báo phương thức
Kế thừa bằng <code>extends</code>	Triển khai bằng <code>implements</code>
Lớp con chỉ kế thừa 1 abstract class	Lớp có thể implements nhiều interface

2.1.1.11 Namespaces (Không gian tên)

- Định nghĩa: Namespaces giúp nhóm các lớp/hàm/hằng số lại với nhau để tránh xung đột tên khi dự án lớn hoặc khi sử dụng nhiều thư viện.
- Cú pháp:

```

// File: App/Models/User.php
namespace App\Models;

class User {

```

```

        public function __construct() {
            echo "Class User trong App\Models";
        }
    }

// File: index.php
require 'App/Models/User.php';
use App\Models\User;

$u = new User();

```

Ghi chú:

- Khai báo namespace phải ở dòng đầu tiên của file PHP.
- Dùng use để nhập (import) lớp từ namespace khác.

2.1.1.12 Autoloading

- Định nghĩa: Autoloading giúp PHP tự động nạp file chứa class khi cần, không phải require thủ công.
- Cách dùng:
 - Tự viết hàm autoload:

```

spl_autoload_register(function ($class) {
    $file = str_replace('\\', '/', $class) . '.php';
    if (file_exists($file)) {
        require $file;
    }
});

$obj = new App\Models\User(); // PHP tự load file

```

- Dùng Composer Autoload:
 - Tạo file composer.json với cấu hình autoload.
 - Chạy composer dump-autoload.
 - Gọi require 'vendor/autoload.php'; để kích hoạt.

Ghi chú:

- Autoload giúp tổ chức mã nguồn gọn gàng và dễ mở rộng.
- PSR-4 là chuẩn phổ biến cho autoloading trong PHP.

2.2 Giới thiệu JavaScript và tương tác với PHP

2.2.1 Giới thiệu

JavaScript (JS) là ngôn ngữ lập trình phía client (trình duyệt), cho phép tạo các trang web tương tác, động và cải thiện trải nghiệm người dùng.

Trong khi **PHP** xử lý logic trên server, **JavaScript** xử lý giao diện và hành vi trên **trình duyệt**.

Sự kết hợp giữa PHP và JavaScript giúp xây dựng **ứng dụng web full-stack**.

2.2.2 JavaScript cơ bản

Chèn JavaScript vào HTML

Có thể đặt JavaScript trong:

- Thẻ `<script>` trực tiếp trong HTML.
- File .js riêng và liên kết bằng src.

Ví dụ:

```
<!-- Cách 1: Nội tuyến -->
<script>
    alert("Xin chào từ JavaScript!");
</script>

<!-- Cách 2: File ngoài -->
<script src="main.js"></script>
```

Biến và kiểu dữ liệu

```
let name = "Nam"; // chuỗi
let age = 20; // số
let isMember = true; // boolean
```

Hàm

```
function greet(user) {
    alert("Xin chào " + user);
}
greet("Nam");
```

Sự kiện (Events)

JavaScript có thể gắn hành vi vào các sự kiện của HTML:

```
<button onclick="alert('Bạn đã bấm nút!')">Bấm tôi</button>
```

DOM Manipulation (Tương tác HTML)

```
<p id="demo">Nội dung ban đầu</p>
<button onclick="document.getElementById('demo').innerHTML = 'Đã thay
đổi';">
    Thay đổi nội dung
</button>
```

2.2.3 Tương tác JavaScript – PHP

JavaScript và PHP không chạy cùng thời điểm:

- PHP chạy trước trên server → trả HTML/JS về trình duyệt.
- JavaScript chạy sau trên client.

Có 2 cách phổ biến để trao đổi dữ liệu:

- Truyền dữ liệu từ PHP sang JavaScript
- Gửi dữ liệu từ JavaScript lên PHP

2.2.3.1 Truyền dữ liệu từ PHP sang JavaScript

- PHP sinh ra mã JavaScript trực tiếp.

```
<?php
$username = "Nguyễn Văn A";
?>
<script>
    let user = "<?php echo $username; ?>";
    alert("Xin chào " + user);
</script>
```

- Hoặc dùng AJAX để tải dữ liệu từ server khi cần.

2.2.3.2 Gửi dữ liệu từ JavaScript lên PHP

Cách 1: Form HTML

```
<form action="process.php" method="POST">
    <input type="text" name="username">
    <button type="submit">Gửi</button>
</form>
```

Cách 2: AJAX (Asynchronous JavaScript and XML)

- AJAX cho phép gửi/nhận dữ liệu với PHP mà không tải lại trang.
- Ví dụ với `fetch()`:

```
fetch("process.php", {
    method: "POST",
    headers: { "Content-Type": "application/x-www-form-urlencoded" },
    body: "name=Nam&age=20"
})
.then(res => res.text())
.then(data => {
    console.log("Phản hồi từ server:", data);
});
```

- File *process.php*:

```
<?php
echo "Xin chào " . $_POST['name'] . ", bạn " . $_POST['age'] . " tuổi.";
?>
```

2.2.4 JSON – Cầu nối dữ liệu giữa PHP và JavaScript

- JSON (JavaScript Object Notation) là định dạng phổ biến để trao đổi dữ liệu.
- PHP hỗ trợ:

```
$data = ["name" => "Nam", "age" => 20];  
echo json_encode($data); // {"name":"Nam","age":20}
```

JavaScript đọc:

```
let obj = JSON.parse('{ "name": "Nam", "age": 20 }');  
console.log(obj.name); // Nam
```

Lưu ý:

- JavaScript xử lý trên trình duyệt, PHP xử lý trên server → không thể gọi PHP trực tiếp từ JS nếu không qua HTTP request.
- AJAX và JSON giúp ứng dụng web mượt hơn, giảm tải server.
- Chú ý bảo mật khi nhận dữ liệu từ JavaScript vì dữ liệu có thể bị chỉnh sửa.

Chương 3. Tương tác cơ sở dữ liệu với MySQL và PDO

Trong các ứng dụng web, việc lưu trữ và quản lý dữ liệu là một phần không thể thiếu. MySQL là hệ quản trị cơ sở dữ liệu mã nguồn mở phổ biến, được sử dụng rộng rãi cho các dự án PHP. Để làm việc với MySQL trong PHP, chúng ta có nhiều cách tiếp cận, trong đó PDO (PHP Data Objects) là phương pháp hiện đại, bảo mật và linh hoạt hơn so với các cách truyền thống như `mysql_*` hay `mysqli_*`.

3.1 Cơ sở dữ liệu MySQL

3.1.1 Tổng quan về MySQL

MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS – Relational Database Management System) mã nguồn mở, sử dụng ngôn ngữ truy vấn có cấu trúc SQL (Structured Query Language) để thao tác dữ liệu. Đây là một trong những hệ quản trị phổ biến nhất hiện nay, được sử dụng rộng rãi trong các ứng dụng web, phần mềm doanh nghiệp, thương mại điện tử và phân tích dữ liệu.

Đặc điểm nổi bật:

- Miễn phí và mã nguồn mở (phiên bản Community Edition).
- Tốc độ xử lý nhanh, khả năng mở rộng cao.
- Hỗ trợ đa nền tảng (Windows, Linux, macOS).
- Tích hợp dễ dàng với nhiều ngôn ngữ lập trình như PHP, Java, Python.
- Cộng đồng người dùng và tài liệu hỗ trợ phong phú.

Ứng dụng thực tế:

- Lưu trữ dữ liệu cho website thương mại điện tử.
- Quản lý thông tin khách hàng (CRM).
- Lưu trữ dữ liệu trong các ứng dụng di động và IoT.

MySQL và MariaDB:

- MySQL: Do Oracle quản lý, phổ biến và được hỗ trợ rộng rãi.
- MariaDB: Một nhánh (fork) từ MySQL, do cộng đồng phát triển, tương thích ngược, hiệu năng và bảo mật được cải thiện.

Cài đặt MySQL Server:

- Windows: Tải từ trang chủ MySQL (<https://dev.mysql.com/downloads/installer/>) hoặc sử dụng gói Laragon/XAMPP/WAMP.
- Linux (Ubuntu/Debian):

```
sudo apt update
sudo apt install mysql-server
```

- macOS: Cài qua Homebrew

```
brew install mysql
```

Công cụ quản lý phổ biến:

- phpMyAdmin: Ứng dụng web quản lý MySQL thông qua trình duyệt.
- SQLyog: Công cụ quản lý MySQL mạnh mẽ trên Windows.
- DBeaver: Trình quản lý cơ sở dữ liệu đa hệ thống, hỗ trợ MySQL và nhiều DBMS khác.

3.1.2 Thiết kế cơ sở dữ liệu

Thiết kế cơ sở dữ liệu là bước quan trọng giúp hệ thống hoạt động hiệu quả và dễ mở rộng.

Khái niệm cơ bản:

- Bảng (Table): Tập hợp dữ liệu được lưu trữ theo hàng và cột.
- Cột (Column): Trường dữ liệu (vd: HọTên, Email, NgàySinh).
- Bản ghi (Record / Row): Một dòng dữ liệu trong bảng.

Khóa chính (Primary Key) và khóa ngoại (Foreign Key):

- Primary Key: Trường/nhóm trường xác định duy nhất mỗi bản ghi.
- Foreign Key: Trường liên kết đến Primary Key của bảng khác.

Các loại mối quan hệ:

- 1 – 1 (One to One): Một bản ghi của bảng A tương ứng với một bản ghi của bảng B.
- 1 – N (One to Many): Một bản ghi của bảng A liên kết với nhiều bản ghi của bảng B.
- N – N (Many to Many): Nhiều bản ghi của bảng A liên kết với nhiều bản ghi của bảng B (thường thông qua bảng trung gian).

Quy tắc đặt tên bảng và cột:

- Tên bảng, cột phải ngắn gọn, dễ hiểu, không chứa ký tự đặc biệt.
- Nên dùng chữ thường và dấu gạch dưới để phân tách từ (vd: user_account).

3.1.3 Quản trị cơ bản

Tạo cơ sở dữ liệu:

```
CREATE DATABASE QuanLySinhVien;
```

Chỉnh sửa cơ sở dữ liệu:

```
ALTER DATABASE QuanLySinhVien CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Xóa cơ sở dữ liệu:

```
DROP DATABASE QuanLySinhVien;
```

Tạo bảng:

```
CREATE TABLE SinhVien (
    MaSV INT PRIMARY KEY,
    HoTen VARCHAR(100),
    NgaySinh DATE
);
```

Chỉnh sửa bảng:


```
ALTER TABLE SinhVien ADD COLUMN Email VARCHAR(100);
```

Xóa bảng:

```
DROP TABLE SinhVien;
```

3.1.4 Sao lưu và phục hồi dữ liệu

Xuất dữ liệu ra file SQL:

```
mysqldump -u root -p QuanLySinhVien > backup.sql
```

Nhập dữ liệu từ file SQL:

```
mysql -u root -p QuanLySinhVien < backup.sql
```

3.2 Ngôn ngữ truy vấn cấu trúc (SQL)

3.2.1 Các câu lệnh cơ bản

Câu lệnh SELECT – Lấy dữ liệu

– Cú pháp:

```
SELECT [cột1], [cột2], ...  
FROM [tên_bảng];
```

– Ví dụ:

```
SELECT id, name, email  
FROM users;
```

Tùy chọn:

- SELECT * → Lấy tất cả các cột.
- DISTINCT → Lọc bỏ giá trị trùng lặp.

Câu lệnh INSERT – Thêm dữ liệu

– Cú pháp:

```
INSERT INTO [tên_bảng] (cột1, cột2, ...)  
VALUES (giá_trị1, giá_trị2, ...);
```

– Ví dụ:

```
INSERT INTO users (name, email, age)  
VALUES ('Nguyễn Văn A', 'a@example.com', 25);
```

Câu lệnh UPDATE – Cập nhật dữ liệu

– Cú pháp:

```
UPDATE [tên_bảng]  
SET cột1 = giá_trị1, cột2 = giá_trị2, ...  
WHERE điều_kiện;
```

- Ví dụ:

```
UPDATE users
SET email = 'new_email@example.com'
WHERE id = 1;
```

Lưu ý: Luôn dùng WHERE để tránh cập nhật toàn bộ bảng.

Câu lệnh DELETE – Xóa dữ liệu

- Cú pháp:

```
DELETE FROM [tên_bảng]
WHERE điều_kiện;
```

- Ví dụ:

```
DELETE FROM users
WHERE id = 5;
```

Lưu ý: Không dùng WHERE sẽ xóa toàn bộ dữ liệu.

3.2.2 Truy vấn nâng cao

Mệnh đề WHERE – Lọc dữ liệu

- Cú pháp:

```
SELECT * FROM [tên_bảng]
WHERE điều_kiện;
```

- Toán tử so sánh: =, <>, <, >, <=, >=
- Toán tử logic: AND, OR, NOT
- Ví dụ:

```
SELECT * FROM users
WHERE age >= 18 AND city = 'Hà Nội';
```

Sắp xếp với ORDER BY

- Cú pháp:

```
SELECT * FROM [tên_bảng]
ORDER BY cột [ASC|DESC];
```

- Ví dụ:

```
SELECT name, age FROM users
ORDER BY age DESC;
```

Gom nhóm với GROUP BY và HAVING

Khái niệm:

- GROUP BY dùng để gom nhóm các bản ghi có giá trị giống nhau ở một hoặc nhiều cột.
- Thường kết hợp với các hàm tổng hợp (Aggregate Functions) để tính toán trên từng nhóm.

- HAVING dùng để lọc kết quả sau khi đã GROUP BY (vì WHERE không thể dùng với kết quả của hàm tổng hợp).

Các hàm tổng hợp phổ biến:

Hàm	Mô tả	Ví dụ
COUNT()	Đếm số lượng bản ghi	COUNT(*), COUNT(column)
SUM()	Tính tổng giá trị	SUM(price)
AVG()	Tính giá trị trung bình	AVG(score)
MIN()	Lấy giá trị nhỏ nhất	MIN(age)
MAX()	Lấy giá trị lớn nhất	MAX(salary)

- Cú pháp:

```
SELECT column1, aggregate_function(column2)
FROM table_name
WHERE condition
GROUP BY column1
HAVING aggregate_function(column2) condition;
```

- Ví dụ:

- Ví dụ 1 – Đếm số sản phẩm theo loại

```
SELECT category_id, COUNT(*) AS total_products
FROM products
GROUP BY category_id;
```

Kết quả: Mỗi category_id kèm theo số sản phẩm thuộc loại đó.

- Ví dụ 2 – Tính tổng doanh thu theo từng khách hàng

```
SELECT customer_id, SUM(amount) AS total_amount
FROM orders
GROUP BY customer_id;
```

Kết quả: Doanh thu của từng khách hàng.

- Ví dụ 3 – Tìm loại hàng có trên 5 sản phẩm (dùng HAVING)

```
SELECT category_id, COUNT(*) AS total_products
FROM products
GROUP BY category_id
HAVING COUNT(*) > 5;
```

HAVING lọc sau khi đã nhóm dữ liệu.

- Ví dụ 4 – Tính lương trung bình của từng phòng ban và lọc ra những phòng ban có lương TB > 10 triệu

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 10000000;
```

Lưu ý:

- GROUP BY luôn đi sau WHERE và trước HAVING.
- Nếu SELECT có cột không dùng trong hàm tổng hợp, cột đó bắt buộc phải nằm trong GROUP BY.
- HAVING có thể dùng chung với WHERE:
 - WHERE lọc bản ghi trước khi nhóm
 - HAVING lọc bản ghi sau khi nhóm

Kết hợp bảng với JOIN

- INNER JOIN – Chỉ lấy các bản ghi khớp ở cả 2 bảng.

```
SELECT u.name, o.order_date
FROM users u
INNER JOIN orders o ON u.id = o.user_id;
```

- LEFT JOIN – Lấy tất cả từ bảng bên trái và dữ liệu khớp từ bảng bên phải.

```
SELECT u.name, o.order_date
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
```

- RIGHT JOIN – Ngược lại với LEFT JOIN.
- FULL JOIN – Lấy tất cả bản ghi từ cả hai bảng (MySQL không hỗ trợ trực tiếp, cần UNION).

Ghi nhớ:

- SQL không phân biệt chữ hoa – thường, nhưng nên viết từ khóa (SELECT, FROM, WHERE) in hoa để dễ đọc.
- Luôn kiểm tra câu lệnh trên một tập dữ liệu nhỏ trước khi áp dụng cho toàn bộ bảng.
- Kết hợp nhiều mệnh đề (WHERE, ORDER BY, GROUP BY, HAVING) để tạo truy vấn linh hoạt.

3.3 Lập trình tương tác cơ sở dữ liệu với PDO

3.3.1 Giới thiệu PDO

PDO (PHP Data Objects) là một thư viện cung cấp giao diện thống nhất để truy cập nhiều loại cơ sở dữ liệu (MySQL, PostgreSQL, SQLite, v.v.).

Ưu điểm so với mysql_ và mysqli_*:*

- Bảo mật: hỗ trợ Prepared Statement, giúp tránh SQL Injection.
- Đa hệ CSDL: chỉ cần thay đổi DSN và driver, không phải viết lại toàn bộ code.
- Tích hợp OOP: dễ quản lý, tái sử dụng code.

- Hỗ trợ xử lý ngoại lệ: thông báo lỗi rõ ràng qua cơ chế try...catch..
- Nhược điểm:* cần cài đặt extension pdo_xxx tương ứng với CSDL.

3.3.2 Kết nối CSDL với PDO

Cấu trúc DSN (Data Source Name):

"mysql:host=localhost;dbname=ten_cSDL;charset=utf8"

- mysql → driver.
- host → địa chỉ máy chủ CSDL.
- dbname → tên cơ sở dữ liệu.
- charset → bộ mã ký tự.

Ví dụ kết nối và xử lý ngoại lệ::

```
<?php
$host = "localhost";
$dbname = "ql_sinhvien";
$username = "root";
$password = "";

try {
    $conn = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8",
$username, $password);
    // Thiết lập chế độ lỗi thành Exception
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Kết nối thành công!";
} catch (PDOException $e) {
    echo "Kết nối thất bại: " . $e->getMessage();
}
```

Giải thích:

- PDO::ATTR_ERRMODE: Đặt chế độ báo lỗi, nên dùng PDO::ERRMODE_EXCEPTION để dễ debug.
- Dùng try...catch để bắt lỗi kết nối.

3.3.3 Thực hiện truy vấn an toàn

Khi làm việc với CSDL, việc truyền trực tiếp dữ liệu người dùng vào câu SQL tiềm ẩn nguy cơ SQL Injection.

Để đảm bảo an toàn, PDO hỗ trợ Prepared Statements – kỹ thuật tách phần câu lệnh SQL và phần dữ liệu đầu vào.

3.3.3.1 Nguyên tắc của Prepared Statements

- Câu lệnh SQL được chuẩn bị trước với các placeholder (biến tạm) để thay thế giá trị thật.
- PDO gửi câu SQL đã chuẩn bị tới CSDL để phân tích cú pháp trước khi gán giá trị.

- Dữ liệu được gán sau (bind) sẽ không bị CSDL hiểu như lệnh SQL, ngăn chặn SQL Injection.

3.3.3.2 Các loại Placeholder trong PDO

PDO hỗ trợ 2 loại placeholder:

- Unnamed placeholder (?) – còn gọi là positional placeholder.
- Named placeholder (:name) – còn gọi là placeholder có tên.

a. Unnamed Placeholder (?)

Cú pháp

- Mỗi dấu ? đại diện cho một giá trị.
- Phải bind giá trị theo đúng thứ tự xuất hiện của ?.
- Đánh số từ 1 khi bind.

Ví dụ:

```
$sql = "SELECT * FROM users WHERE username = ? AND status = ?";
$stmt = $pdo->prepare($sql);

$stmt->bindValue(1, 'admin', PDO::PARAM_STR);
$stmt->bindValue(2, 1, PDO::PARAM_INT);

$stmt->execute();
$data = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Ưu điểm:

- Ngắn gọn, dễ viết khi số tham số ít.

Nhược điểm:

- Khó bảo trì khi nhiều tham số vì phải nhớ đúng thứ tự.

b. Named Placeholder (:name)

Cú pháp

- Placeholder có dạng :tên, ví dụ :username, :status.
- Không phụ thuộc vào thứ tự, bind theo tên.

Ví dụ:

```
$sql = "SELECT * FROM users WHERE username = :username AND status = :status";
$stmt = $pdo->prepare($sql);

$stmt->bindValue(':username', 'admin', PDO::PARAM_STR);
$stmt->bindValue(':status', 1, PDO::PARAM_INT);

$stmt->execute();
$data = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Ưu điểm:

- Dễ đọc, dễ bảo trì.
- Phù hợp cho câu SQL dài hoặc nhiều tham số.

Nhược điểm:

- Viết dài hơn so với ?.

c. Bảng so sánh ? và :name

Tiêu chí	? (Unnamed)	:name (Named)
Độ rõ ràng	Thấp – phải dựa vào thứ tự	Cao – tên tham số rõ ràng
Số lượng tham số	Tốt cho ít tham số	Tốt cho nhiều tham số
Thứ tự bind	Bắt buộc đúng thứ tự	Không cần đúng thứ tự
Cú pháp bind	<code>bindValue(1, 'value')</code>	<code>bindValue(':name', 'value')</code>
Khả năng bảo trì	Khó khi thêm/bớt tham số	Dễ hơn vì chỉ cần thêm tên mới
Tình huống khuyên dùng	Truy vấn đơn giản, ít tham số	Truy vấn phức tạp, dự án lớn

3.3.3.3 Lưu ý khi sử dụng

- Không bao giờ nối chuỗi dữ liệu người dùng trực tiếp vào câu SQL.
- Có thể dùng `bindParam()` (tham chiếu) hoặc `bindValue()` (giá trị) để gán.
- Khi dữ liệu chỉ dùng một lần và không thay đổi → dùng `bindValue()` sẽ gọn hơn.
- Khi dữ liệu thay đổi giá trị trong vòng lặp → dùng `bindParam()`.

3.3.3.4 Ví dụ tổng hợp

a. Dùng ?

```
$sql = "UPDATE products SET name = ?, price = ? WHERE id = ?";
$stmt = $pdo->prepare($sql);
$stmt->bindValue(1, 'Bàn gỗ', PDO::PARAM_STR);
$stmt->bindValue(2, 2500000, PDO::PARAM_INT);
$stmt->bindValue(3, 5, PDO::PARAM_INT);
$stmt->execute();
```

b. Dùng :name

```
$sql = "UPDATE products SET name = :name, price = :price WHERE id = :id";
$stmt = $pdo->prepare($sql);
$stmt->bindValue(':name', 'Bàn gỗ', PDO::PARAM_STR);
$stmt->bindValue(':price', 2500000, PDO::PARAM_INT);
$stmt->bindValue(':id', 5, PDO::PARAM_INT);
$stmt->execute();
```

3.3.4 Các thao tác CRUD với PDO

a. SELECT – Lấy dữ liệu

Sử dụng `query()`

```
$sql = "SELECT id, hoten, ngaysinh FROM sinhvien";
$stmt = $pdo->query($sql);

foreach ($stmt as $row) {
    echo $row['id'] . " - " . $row['hoten'] . " - " . $row['ngaysinh'] .
"<br>";
}
```

- `query()` dùng cho câu lệnh không có tham số.
- Trả về `PDOStatement`, có thể duyệt qua như mảng.

Sử dụng `fetchAll()`

Không điều kiện

```
$sql = "SELECT * FROM sinhvien";
$stmt = $conn->prepare($sql);
$stmt->execute();
$data = $stmt->fetchAll(PDO::FETCH_ASSOC);
```

Có điều kiện

```
$sql = "SELECT * FROM sinhvien WHERE hoten LIKE :ten";
$stmt = $pdo->prepare($sql);
$stmt->execute(['ten' => '%Nam%']);

$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
foreach ($rows as $row) {
    echo $row['hoten'] . "<br>";
}
```

Lưu ý: Trong `execute()` của PDO, khi truyền một mảng, PDO sẽ tự động thêm dấu ":" vào key nếu nó chưa có. Ở ví dụ trên, hàm `execute(['ten' => '%Nam%'])` tương đương với `execute([':ten' => '%Nam%'])`

b. INSERT – Thêm dữ liệu

```
$sql = "INSERT INTO sinhvien (hoten, lop) VALUES (:hoten, :lop)";
$stmt = $conn->prepare($sql);
$stmt->bindValue(':hoten', 'Nguyễn Văn A');
$stmt->bindValue(':lop', 'CNTT1');
$stmt->execute();
```

c. UPDATE – Cập nhật dữ liệu

```
$sql = "UPDATE sinhvien SET hoten = :hoten WHERE id = :id";
$stmt = $conn->prepare($sql);
$stmt->execute([':hoten' => 'Trần Thị B', ':id' => 1]);
```


d. DELETE – Xóa dữ liệu

```
$sql = "DELETE FROM sinhvien WHERE id = :id";  
$stmt = $conn->prepare($sql);  
$stmt->execute([':id' => 3]);
```

e. Hiển thị dữ liệu ra HTML Table

```
<table border="1">  
  <tr>  
    <th>ID</th>  
    <th>Họ tên</th>  
    <th>Lớp</th>  
  </tr>  
  <?php foreach ($data as $row): ?>  
    <tr>  
      <td><?= $row['id'] ?></td>  
      <td><?= htmlspecialchars($row['hoten']) ?></td>  
      <td><?= htmlspecialchars($row['lop']) ?></td>  
    </tr>  
  <?php endforeach; ?>  
</table>
```

- htmlspecialchars() giúp tránh XSS khi hiển thị dữ liệu từ DB.

3.4 Các kỹ thuật hiển thị dữ liệu nâng cao

3.4.1 Phân trang (Pagination)

a. Ý nghĩa của phân trang

- Khi dữ liệu trong bảng quá lớn (hàng ngàn hoặc hàng triệu bản ghi), việc hiển thị toàn bộ sẽ:
 - Gây chậm hệ thống.
 - Khó đọc và tìm kiếm.
 - Tốn băng thông.
- Phân trang giúp:
 - Hiển thị dữ liệu theo từng trang nhỏ.
 - Giảm tải cho server và client.
 - Cải thiện trải nghiệm người dùng.

b. Cách tính số trang và giới hạn dữ liệu

- Công thức tính số trang:

```
tổng_số_trang = ceil(tổng_số_bản_ghi / số_bản_ghi_mỗi_trang)
```

- Các tham số quan trọng:
 - \$limit: số bản ghi trên mỗi trang (ví dụ: 10).
 - \$page: số trang hiện tại (lấy từ \$_GET['page']).

- \$offset: vị trí bắt đầu lấy dữ liệu.

```
$offset = ($page - 1) * $limit;
```

c. Triển khai phân trang với SQL (LIMIT, OFFSET)

Ví dụ: Lấy danh sách sinh viên với phân trang:

```
<?php
$limit = 10; // Số bản ghi mỗi trang
$page = isset($_GET['page']) ? (int)$_GET['page'] : 1;
$offset = ($page - 1) * $limit;

// Kết nối PDO
$conn = new PDO("mysql:host=localhost;dbname=qlsv", "root", "");

// Lấy tổng số bản ghi
$totalStmt = $conn->query("SELECT COUNT(*) FROM sinhvien");
$totalRecords = $totalStmt->fetchColumn();
$totalPages = ceil($totalRecords / $limit);

// Lấy dữ liệu cho trang hiện tại
$stmt = $conn->prepare("SELECT * FROM sinhvien LIMIT :limit OFFSET :offset");
$stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
$stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
$stmt->execute();
$students = $stmt->fetchAll(PDO::FETCH_ASSOC);
?>
```

3.4.2 Tìm kiếm (Searching)

a. Tìm kiếm toàn bộ bảng với LIKE

Cú pháp:

```
SELECT * FROM bảng WHERE cột LIKE '%từ_khóa%';
```

Ví dụ tìm sinh viên có tên chứa chữ "An":

```
$keyword = isset($_GET['keyword']) ? $_GET['keyword'] : '';
$sql = "SELECT * FROM sinhvien WHERE hoten LIKE :keyword";
$stmt = $conn->prepare($sql);
$stmt->bindValue(':keyword', '%' . $keyword . '%', PDO::PARAM_STR);
$stmt->execute();
```

b. Tìm kiếm nâng cao theo nhiều điều kiện

Có thể tìm theo tên và lớp cùng lúc:

```
$keyword = $_GET['keyword'] ?? '';
$class = $_GET['class'] ?? '';

$sql = "SELECT * FROM sinhvien WHERE 1=1";
```

```

if (!empty($keyword)) {
    $sql .= " AND hoten LIKE :keyword";
}
if (!empty($class)) {
    $sql .= " AND malop = :class";
}

$stmt = $conn->prepare($sql);

if (!empty($keyword)) {
    $stmt->bindValue(':keyword', '%' . $keyword . '%');
}
if (!empty($class)) {
    $stmt->bindValue(':class', $class);
}

$stmt->execute();

```

c. Kết hợp phân trang và tìm kiếm

Khi kết hợp, cần giữ nguyên tham số tìm kiếm khi chuyển trang.

Ví dụ:

```

$sql = "SELECT * FROM sinhvien WHERE hoten LIKE :keyword LIMIT :limit
OFFSET :offset";
$stmt = $conn->prepare($sql);
$stmt->bindValue(':keyword', '%' . $keyword . '%', PDO::PARAM_STR);
$stmt->bindValue(':limit', $limit, PDO::PARAM_INT);
$stmt->bindValue(':offset', $offset, PDO::PARAM_INT);
$stmt->execute();

```

Khi hiển thị liên kết phân trang:

```

for ($i = 1; $i <= $totalPages; $i++) {
    echo "<a href='?page=$i&keyword=$keyword'>$i</a> ";
}

```

Chương 4. Phát triển ứng dụng web với Laravel Framework

Chương này tập trung giới thiệu và hướng dẫn sinh viên từng bước phát triển ứng dụng web với Laravel. Nội dung bao gồm: làm quen với kiến trúc và môi trường phát triển của Laravel; xây dựng chức năng xử lý request thông qua Routing, Controller và View; khai thác Eloquent ORM để quản lý cơ sở dữ liệu một cách trực quan; áp dụng các tính năng cốt lõi như Validation, Middleware, Authentication và Artisan CLI. Bên cạnh đó, sinh viên sẽ được thực hành xây dựng các ứng dụng web cơ bản như website tin tức, website bán hàng đơn giản, và tìm hiểu quy trình đóng gói, triển khai ứng dụng lên môi trường thực tế.

4.1 Giới thiệu Laravel Framework

4.1.1 Tổng quan về Framework: Lợi ích của việc sử dụng framework

Framework trong lập trình web là một bộ công cụ, thư viện và quy tắc được thiết kế sẵn, hỗ trợ lập trình viên phát triển ứng dụng một cách nhanh chóng và có hệ thống. Thay vì phải xây dựng mọi thứ từ đầu, framework cung cấp các thành phần đã chuẩn hóa như quản lý routing, kết nối cơ sở dữ liệu, xác thực người dùng, hoặc xử lý bảo mật.

Lợi ích khi sử dụng framework:

- *Tăng tốc độ phát triển*: Giúp giảm thiểu việc viết lại các đoạn mã lặp đi lặp lại.
- *Tính bảo mật cao*: Cung cấp sẵn nhiều cơ chế bảo mật chống SQL Injection, XSS, CSRF.
- *Dễ bảo trì và mở rộng*: Cấu trúc dự án rõ ràng, giúp nhiều lập trình viên có thể cùng làm việc.
- *Tính cộng đồng*: Các framework phổ biến như Laravel có cộng đồng lớn, tài liệu phong phú và nhiều gói thư viện mở rộng.

4.1.2 Kiến trúc MVC (Model – View – Controller) trong Laravel

Laravel tuân theo mô hình kiến trúc **MVC**, giúp phân tách rõ ràng giữa ba thành phần chính:

- *Model*: Đại diện cho dữ liệu và các quy tắc nghiệp vụ liên quan. Trong Laravel, Model thường kết hợp với Eloquent ORM để thao tác với cơ sở dữ liệu.
- *View*: Chịu trách nhiệm hiển thị dữ liệu và giao diện người dùng. Laravel sử dụng Blade Template Engine để xây dựng View.
- *Controller*: Đóng vai trò trung gian, nhận request từ người dùng, xử lý nghiệp vụ thông qua Model, sau đó trả kết quả cho View.

Nhờ **MVC**, ứng dụng trở nên dễ dàng bảo trì, kiểm thử và tái sử dụng mã nguồn.

4.1.3 Cài đặt Laravel

a. Cài đặt Laravel bằng Composer trực tiếp

Để khởi tạo một dự án Laravel, thông thường, chúng ta sử dụng công cụ **Composer** (trình quản lý gói cho PHP). Các bước cơ bản gồm:

1. Cài đặt Composer trên hệ thống từ getcomposer.org
2. Chạy lệnh khởi tạo dự án:

```
composer create-project laravel/laravel ten_du_an
```

3. Cấu hình môi trường trong file .env (thiết lập thông số kết nối cơ sở dữ liệu, cổng chạy ứng dụng, bảo mật...).
4. Khởi động máy chủ nội bộ bằng lệnh:

```
php artisan serve
```

Kết quả, ứng dụng Laravel cơ bản sẽ sẵn sàng truy cập tại địa chỉ <http://localhost:8000>.

Lưu ý: Nếu chúng ta đã cài đặt Laragon, CLI của Composer có thể sử dụng trực tiếp trong Terminal của Laragon, và không cần phải cài đặt riêng.

b. Cài đặt Laravel sử dụng Laragon

Laragon là công cụ tất cả-trong-một (Apache/Nginx, PHP, MySQL, phpMyAdmin) giúp cài đặt Laravel nhanh hơn, ít thao tác hơn. Quy trình:

1. Cài đặt Laragon từ laragon.org.
2. Mở Laragon, chọn Menu → Quick app → Laravel.
3. Đặt tên dự án, Laragon sẽ tự động tải về và cấu hình Laravel.
4. Sau vài phút, dự án sẵn sàng và có thể truy cập ngay tại http://ten_du_an.test nhờ hệ thống Virtual Host của Laragon.
5. Toàn bộ môi trường MySQL, PHP, phpMyAdmin đã được cấu hình sẵn, sinh viên chỉ cần tập trung phát triển ứng dụng.

4.1.4 Cấu trúc thư mục của một dự án Laravel

Một dự án Laravel được tổ chức với cấu trúc thư mục khoa học. Một số thư mục quan trọng:

- app/: Chứa các thành phần cốt lõi của ứng dụng như Models, Controllers, Middleware.
- resources/views/: Chứa các file Blade (View) để hiển thị giao diện người dùng.
- routes/: Định nghĩa hệ thống routing (web.php, api.php).
- database/: Quản lý Migration, Seeder và Factory.
- public/: Chứa tài nguyên công khai như hình ảnh, file CSS, JavaScript.
- config/: Chứa các file cấu hình hệ thống.

Việc hiểu rõ cấu trúc này giúp sinh viên dễ dàng định vị các thành phần, từ đó phát triển ứng dụng một cách mạch lạc và hiệu quả.

4.2 Routing, Controller và View trong Laravel

Trong kiến trúc MVC của **Laravel**, ba thành phần **Routing** – **Controller** – **View** đóng vai trò trung tâm trong xử lý yêu cầu và hiển thị nội dung cho người dùng. Quy trình hoạt động cơ bản: người dùng gửi yêu cầu thông qua trình duyệt → hệ thống định tuyến (**Routing**) phân tích yêu cầu → **Controller** xử lý logic nghiệp vụ → dữ liệu được truyền đến **View** để hiển thị.

4.2.1 Routing trong Laravel

Routing dùng để định nghĩa các đường dẫn (URL) và gắn chúng với chức năng xử lý tương ứng. Laravel quản lý hệ thống route thông qua các tệp trong thư mục routes/, trong đó web.php thường được sử dụng cho ứng dụng web và api.php cho dịch vụ API.

Một số ví dụ minh họa:

```
// Route cơ bản
Route::get('/hello', function () {
    return 'Xin chào Laravel!';
});

// Route với tham số động
Route::get('/user/{id}', function ($id) {
    return "Người dùng có ID: " . $id;
});

// Route có tên (named route)
Route::get('/profile', [UserController::class, 'show'])->
    name('profile');
```

Laravel hỗ trợ nhiều phương thức HTTP khác nhau như get, post, put, delete. Ngoài ra, các route có thể được nhóm lại để dễ quản lý và áp dụng chung middleware, chẳng hạn như kiểm tra đăng nhập trước khi truy cập.

4.2.2 Controller trong Laravel

Controller là lớp chịu trách nhiệm xử lý logic nghiệp vụ. Thay vì viết logic trực tiếp trong route, ta tách riêng ra thành Controller để mã nguồn dễ quản lý và tái sử dụng.

Tạo một Controller mới bằng lệnh Artisan:

```
php artisan make:controller UserController
```

Ví dụ một Controller đơn giản:

```
namespace App\Http\Controllers;

use Illuminate\Http\Request;

class UserController extends Controller
{
    public function index() {
        return "Danh sách người dùng";
    }

    public function show($id) {
        return "Chi tiết người dùng có ID: " . $id;
    }
}
```

Để liên kết Controller với Route:

```
Route::get('/users', [UserController::class, 'index']);
Route::get('/users/{id}', [UserController::class, 'show']);
```

Laravel cũng hỗ trợ *Resource Controller*, cung cấp sẵn các phương thức chuẩn để thực hiện CRUD (tạo, đọc, cập nhật, xóa), giúp phát triển nhanh các ứng dụng web.

4.2.3 View và Blade Template Engine

View là thành phần hiển thị dữ liệu ra cho người dùng, thường được lưu trong thư mục *resources/views/*. Laravel sử dụng Blade Template Engine để xây dựng giao diện với cú pháp ngắn gọn và dễ hiểu.

Ví dụ một View đơn giản (*welcome.blade.php*):

```
<!DOCTYPE html>
<html>
<head>
    <title>Trang chào mừng</title>
</head>
<body>
    <h1>Xin chào, {{ $name }}</h1>
</body>
</html>
```

Controller có thể trả về View kèm dữ liệu:

```
public function welcome()
{
    return view('welcome', ['name' => 'Sinh viên CNTT']);
}
```

Blade hỗ trợ nhiều tính năng hữu ích:

a. Cú pháp hiển thị dữ liệu

Hiển thị biến:

```
{{ $variable }}
```

Mặc định sẽ được escape (chống XSS). Nếu muốn hiển thị HTML thô:

```
{!! $htmlContent !!}
```

b. Cấu trúc điều khiển:

Blade hỗ trợ đầy đủ các câu lệnh điều khiển:

- Câu lệnh điều kiện:

```
@if($age >= 18)
    <p>Bạn đã đủ tuổi.</p>
@endif
@elseif($age >= 16)
    <p>Bạn cần có sự giám hộ.</p>
@else
    <p>Bạn chưa đủ tuổi.</p>
@endif
```

- Câu lệnh switch:

```
@switch($role)
    @case('admin')
        <p>Quản trị viên</p>
    @default
        <p>Người dùng</p>
@endswitch
```

```

        @break
    @case('user')
        <p>Người dùng</p>
    @break
    @default
        <p>Khách</p>
@endswitch

```

c. Vòng lặp:

Blade cung cấp các vòng lặp tương tự PHP:

– foreach:

```

@foreach($users as $user)
    <li>{{ $user->name }}</li>
@endforeach

```

– for:

```

@for($i = 0; $i < 5; $i++)
    <p>Giá trị: {{ $i }}</p>
@endfor

```

– while:

```

@while($count < 10)
    <p>Đếm: {{ $count }}</p>
    @php $count++; @endphp
@endwhile

```

Ngoài ra, Blade hỗ trợ \$loop trong vòng lặp để truy cập thông tin:

```

@foreach($users as $user)
    <p>{{ $loop->iteration }} - {{ $user->name }}</p>
@endforeach

```

d. Template Inheritance (Kế thừa giao diện)

Đây là tính năng mạnh mẽ nhất của Blade, cho phép xây dựng layout chính và các view con kế thừa.

– Layout chính (resources/views/layouts/app.blade.php):

```

<html>
<head>
    <title>@yield('title')</title>
</head>
<body>
    <div class="container">
        @yield('content')
    </div>
</body>
</html>

```


- View con kế thừa (*resources/views/home.blade.php*):

```
@extends('layouts.app')

@section('title', 'Trang chủ')

@section('content')
    <h1>Chào mừng đến với Laravel</h1>
@endsection
```

- @include: Nhúng một template con (thường là header, footer, menu).

```
@include('partials.header')
```

e. Blade Components

Từ Laravel 7 trở đi, Blade hỗ trợ cú pháp component hiện đại <x-...>.

- Tạo component:

```
php artisan make:component Alert
```

Đoạn code trên sẽ sinh ra:

- *app/View/Components/Alert.php*

```
<?php

namespace App\View\Components;

use Closure;
use Illuminate\Contracts\View\View;
use Illuminate\View\Component;

class Alert extends Component
{
    /**
     * Create a new component instance.
     */
    public function __construct()
    {
        //
    }

    /**
     * Get the view / contents that represent the component.
     */
    public function render(): View|Closure|string
    {
        return view('components.alert');
    }
}
```

- `resources/views/components/alert.blade.php`

```
<div>
    <!-- I have not failed. I've just found 10,000 ways that won't work.
    - Thomas Edison -->
</div>
```

Để truyền đối số vào component, cần điều chỉnh:

- Hàm khởi tạo component

```
public function __construct(
    public string $type,
    public string $title
) {
    //
}
```

- Truyền lên giao diện component

```
<div class="alert alert-{{ $type }}">
    <strong>{{ $title }}</strong> - {{ $slot }}
</div>
```

- Sử dụng trong view:

```
<x-alert type="danger" title="Lỗi">
    Có sự cố xảy ra, vui lòng thử lại.
</x-alert>
```

- Anonymous component (không cần class):

- Tạo trực tiếp `resources/views/components/button.blade.php`:

```
<button class="btn btn-{{ $type }}">{{ $slot }}</button>
```

- Gọi trong view:

```
<x-button type="primary">Nhấn vào đây</x-button>
```

f. Các directive tiện ích

Laravel hỗ trợ các cú pháp gọn gàng cho các thuộc tính HTML động:

```
<input type="checkbox" @checked($user->active)>
<option @selected($item->id == $selectedId)>{{ $item->name }}</option>
<input type="text" @disabled($isLocked)>

<span @class([
    'text-green-500' => $active,
    'text-gray-500' => !$active
])>
    Trạng thái
</span>
```

g. Các directive khác

- @csrf: Sinh token CSRF cho form.
- @method('PUT'): Giả lập phương thức HTTP ngoài GET/POST.
- @error('field'): Hiển thị lỗi xác thực.

```
@error('email')
    <div class="alert">{{ $message }}</div>
@enderror
```

- @once: Render nội dung một lần duy nhất.
- @push và @stack: Quản lý chèn nội dung động vào layout.
- @inject: Tiêm service/container vào view.
- @verbatim: Bỏ qua xử lý Blade, giữ nguyên nội dung.

Bảng tổng hợp gợi nhớ – Blade Template Engine

Nhóm chức năng	Directive / Cú pháp	Ví dụ sử dụng
Hiển thị dữ liệu	{{ \$var }}	{{ \$name }} → escape HTML
	{{!! \$html !!}}	Hiển thị HTML thô, không escape
Điều kiện	@if ... @elseif ... @else ... @endif	@if(\$age >= 18)...@endif
	@switch ... @case ... @break ... @endswitch	@switch(\$role)...@endswitch
Vòng lặp	@foreach ... @endforeach	@foreach(\$users as \$u) {{ \$u->name }} @endforeach
	@for ... @endfor	@for(\$i=0;\$i<5;\$i++) {{ \$i }} @endfor
	@while ... @endwhile	@while(\$count<10)...@endwhile
	\$loop	{{ \$loop->iteration }}
Layout & include	@extends('layout')	Kế thừa layout
	@section('content') ... @endsection	Khai báo nội dung section
	@yield('content')	Nơi hiển thị nội dung section
	@include('partials.header')	Chèn view phụ

Nhóm chức năng	Directive / Cú pháp	Ví dụ sử dụng
Blade Components	<code><x-alert type="danger" title="Lỗi">...</x-alert></code>	Gọi component <code><x-...></code> mới (thay <code>@component</code>)
	<code><x-button type="primary">Luu</x-button></code>	Anonymous component
Form & xác thực	<code>@csrf</code>	Sinh token bảo mật cho form
	<code>@method('PUT')</code>	Giả lập HTTP method
	<code>@error('field') ... @enderror</code>	Hiển thị lỗi xác thực cho trường dữ liệu
Thuộc tính động	<code>@checked(\$expr)</code>	<code><input type="checkbox" @checked(\$isActive)></code>
	<code>@selected(\$expr)</code>	<code><option @selected(\$id==\$selected)>...</option></code>
	<code>@disabled(\$expr)</code>	<code><input type="text" @disabled(\$locked)></code>
	<code>@class([...])</code>	<code>\$ok, 'red'=> !\$ok])></code>
Nâng cao	<code>@once ... @endonce</code>	Render một lần duy nhất
	<code>@push('scripts') ... @endpush + @stack</code>	Quản lý nội dung động (JS, CSS)
	<code>@inject('service', 'App\Services\X')</code>	Tiêm service/container vào view
	<code>@verbatim ... @endverbatim</code>	Bỏ qua Blade, giữ nguyên nội dung HTML/JS

4.3 Eloquent ORM và tương tác cơ sở dữ liệu

Trong Laravel, việc làm việc với cơ sở dữ liệu được hỗ trợ mạnh mẽ thông qua Eloquent ORM. Đây là một công cụ ánh xạ giữa bảng trong cơ sở dữ liệu quan hệ và mô hình hướng đối tượng trong PHP, giúp thao tác dữ liệu đơn giản, trực quan và dễ bảo trì hơn so với việc viết thuần SQL.

4.3.1 Eloquent ORM: Tương tác cơ sở dữ liệu hướng đối tượng

ORM (Object-Relational Mapping): ánh xạ giữa bảng dữ liệu và lớp trong ngôn ngữ lập trình.

Eloquent ORM: ORM mặc định trong Laravel, cho phép:

- Lấy dữ liệu (all, find, where, first...),
- Tạo mới, cập nhật, xóa bản ghi,
- Định nghĩa và khai thác quan hệ giữa các bảng (1-1, 1-nhiều, nhiều-nhiều).

Ví dụ:

```
// Lấy tất cả bản ghi
$users = User::all();

// Lấy một bản ghi theo ID
$user = User::find(1);

// Thêm bản ghi mới
$user = new User();
$user->name = "Nguyễn Văn A";
$user->email = "vana@example.com";
$user->save();
```

4.3.2 Cấu hình và kết nối MySQL với Eloquent

1. Tạo cơ sở dữ liệu rỗng (ví dụ huit_oss) trong MySQL.
2. Khai báo trong file .env:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=huit_oss
DB_USERNAME=root
DB_PASSWORD=
```

3. Laravel tự động ánh xạ cấu hình trong config/database.php, đọc từ .env.
4. Kiểm tra kết nối bằng cách chạy migration:

```
php artisan migrate
```

4.3.3 Migration: Quản lý cấu trúc cơ sở dữ liệu bằng mã nguồn

Migration là “lược sử” của cơ sở dữ liệu, giúp dễ dàng tạo, thay đổi hoặc phục hồi cấu trúc.

- Tạo Migration:

```
php artisan make:migration create_products_table
```

- Ví dụ định nghĩa bảng:

```
public function up(): void
{
    Schema::create('products', function (Blueprint $table) {
        $table->id();
        $table->string('name');
```

```

        $table->decimal('price', 10, 2);
        $table->timestamps();
    });
}

```

- Thao tác Migration:

```

php artisan migrate          // chạy migration
php artisan migrate:rollback // quay lui
php artisan migrate:fresh    // xóa và chạy lại toàn bộ

```

4.3.4 Seeder và Factory: Tạo dữ liệu mẫu

Để phục vụ phát triển và kiểm thử, Laravel hỗ trợ tạo dữ liệu giả.

- Seeder:

```
php artisan make:seeder ProductSeeder
```

database\seeders\ProductSeeder.php

```

class ProductSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        DB::table('products')->insert([
            'name' => 'Điện thoại iPhone',
            'price' => 25000000,
        ]);
    }
}

```

- Tạo Model và Factory:

```
php artisan make:model Product --factory
```

app\Models\Product.php

```

class Product extends Model
{
    /** @use HasFactory<\Database\Factories\ProductFactory> */
    use HasFactory;
    protected $fillable = ['name', 'price'];
}

```

database\Factories\ProductFactory.php

```

class ProductFactory extends Factory
{

```

```

/**
 * Define the model's default state.
 *
 * @return array<string, mixed>
 */
public function definition(): array
{
    return [
        'name' => fake()->word(),
        'price' => fake()->numberBetween(1000000, 50000000),
    ];
}
}

```

- Chạy Seeder/Factory:

```

php artisan db:seed --class=ProductSeeder
php artisan tinker
> App\Models\Product::factory()->count(10)->create();

```

4.3.5 Các mối quan hệ trong Eloquent

Laravel cho phép định nghĩa quan hệ trực tiếp trong Model:

- Một – Một (One to One):

```

public function profile()
{
    return $this->hasOne(Profile::class);
}

```

- Một – Nhiều (One to Many):

```

public function products()
{
    return $this->hasMany(Product::class);
}

```

- Nhiều – Nhiều (Many to Many):

```

public function courses()
{
    return $this->belongsToMany(Course::class);
}

```

- Eager Loading:

```

$users = User::with('profile')->get();

```

4.3.6 Hiển thị dữ liệu với Eloquent

Eloquent kết hợp với Controller và Blade để đưa dữ liệu ra giao diện:

- Controller:

```
public function index()
{
    $products = Product::paginate(10);
    return view('products.index', compact('products'));
}
```

- View:

```
@foreach($products as $product)
    <p>{{ $product->name }} - {{ number_format($product->price) }} VNĐ</p>
@endforeach

{{ $products->links() }} <!-- Phân trang -->
```

4.4 Các tính năng cơ bản khác trong Laravel

Ngoài các thành phần nền tảng như Routing, Controller, View và Eloquent ORM, Laravel còn cung cấp nhiều tính năng quan trọng khác giúp xây dựng ứng dụng web an toàn, hiệu quả và dễ mở rộng. Bốn tính năng tiêu biểu gồm: *Validation*, *Middleware*, *Authentication & Authorization*, *Artisan CLI*.

4.4.1 Validation – Kiểm tra và xác thực dữ liệu đầu vào

Khái niệm: Validation giúp kiểm tra dữ liệu mà người dùng nhập (qua form, API request...) trước khi xử lý. Laravel cung cấp sẵn nhiều quy tắc xác thực (validation rules) và thông báo lỗi tùy biến.

Ví dụ trong Controller:

```
public function store(Request $request)
{
    $validated = $request->validate([
        'name' => 'required|string|max:255',
        'email' => 'required|email|unique:users',
        'age' => 'nullable|integer|min:18',
    ]);

    User::create($validated);
    return redirect()->back()->with('success', 'Thêm người dùng thành công');
}
```

Hiển thị lỗi trong View (Blade):

```
@error('email')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror
```


4.4.2 Middleware – Xử lý request trước khi đến Controller

Khái niệm: Middleware là lớp trung gian lọc và xử lý HTTP request/response: xác thực đăng nhập, chống CSRF, ghi log, giới hạn quyền truy cập,... Laravel cung cấp sẵn nhiều middleware (ví dụ auth, verified, throttle), đồng thời cho phép định nghĩa middleware tùy biến.

Tạo Middleware mới:

```
php artisan make:middleware CheckAdmin
```

Lệnh tạo file trong `app/Http/Middleware/CheckAdmin.php`.

Ví dụ Middleware:

```
public function handle($request, Closure $next)
{
    if (!auth()->user() || !auth()->user()->is_admin) {
        return redirect('/home');
    }
    return $next($request);
}
```

Đăng ký & cấu hình middleware trong Laravel: Từ Laravel 11, toàn bộ cấu hình middleware đặt tại `bootstrap/app.php` thông qua callback `withMiddleware()`; không còn (hoặc không cần) chỉnh trong `App\Http\Kernel`. Ta có thể:

- Đặt alias (bí danh) cho middleware để dùng ngắn gọn trong route.
- Thêm/bớt middleware toàn cục (global) hoặc theo nhóm (web, api).
- Cấu hình ngoại lệ *CSRF*, *prepend/append* vào nhóm,...

Mẫu cấu hình:

```
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use App\Http\Middleware\CheckAdmin;

return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        // 2.1 Alias cho route
        $middleware->alias([
            'admin' => CheckAdmin::class,
            // ví dụ: 'subscribed' =>
            \App\Http\Middleware\EnsureUserIsSubscribed::class,
        ]);

        // 2.2 Global middleware (thêm vào cuối chuỗi)
```

```

        // $middleware->append(\App\Http\Middleware\LogRequest::class);

        // 2.3 Thêm vào nhóm 'web' / 'api'
        $middleware->appendToGroup('web',
\App\Http\Middleware\AddSecurityHeaders::class);
        // $middleware->prependToGroup('api',
\App\Http\Middleware\BeforeRateLimit::class);
        // $middleware->removeFromGroup('web',
\App\Http\Middleware\SomeOldMiddleware::class);

        // 2.4 CSRF: loại trừ một số đường dẫn
        $middleware->validateCsrfTokens(except: ['stripe/*']);
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();

```

Các tác vụ trên phản ánh đúng cơ chế cấu hình mới của Laravel: alias, append / prepend / remove theo group, và CSRF exceptions, tất cả tập trung tại bootstrap/app.php

Áp dụng middleware trong route

```

use App\Http\Controllers\AdminController;
use Illuminate\Support\Facades\Route;

Route::middleware(['web', 'auth', 'admin'])
    ->group(function () {
        Route::get('/dashboard', [AdminController::class, 'index']);
    });

```

Hoặc cũng có thể gán trực tiếp trên route đơn lẻ:

```

Route::get('/reports', [AdminController::class, 'reports'])
    ->middleware('admin');

```

4.4.3 Authentication và Authorization cơ bản

a. Authentication (Xác thực người dùng)

- Laravel hỗ trợ sẵn hệ thống đăng nhập, đăng ký, quên mật khẩu.
- Phiên bản Laravel 12 sử dụng package Laravel Breeze hoặc Laravel UI để tạo nhanh scaffolding.

Cài đặt Laravel Breeze:

```

composer require laravel/breeze --dev
php artisan breeze:install
npm install && npm run dev
php artisan migrate

```

➔ Hệ thống đăng nhập/đăng ký cơ bản sẵn sàng.

b. Authorization (Phân quyền người dùng)

Laravel hỗ trợ Gate và Policy để kiểm soát quyền truy cập.

Ví dụ Gate:

```
Gate::define('edit-post', function ($user, $post) {  
    return $user->id === $post->user_id;  
});
```

Áp dụng trong Controller:

```
if (Gate::allows('edit-post', $post)) {  
    // Cho phép chỉnh sửa  
}
```

Sử dụng trong Blade:

```
@can('edit-post', $post)  
    <a href="{ route('posts.edit', $post) }}">Chỉnh sửa</a>  
@endcan
```

4.4.4 Artisan CLI – Công cụ dòng lệnh

Khái niệm: Artisan là công cụ dòng lệnh mạnh mẽ đi kèm Laravel, giúp tự động hóa nhiều thao tác (tạo model, controller, migration, chạy seeder, dọn cache...).

Một số lệnh phổ biến:

```
php artisan list                # Liệt kê tất cả lệnh  
php artisan make:model Product -m # Tạo model kèm migration  
php artisan make:controller UserController  
php artisan make:seeder ProductSeeder  
php artisan migrate:fresh --seed  # Làm mới DB và seed lại dữ liệu  
php artisan db:seed              # chạy seeder  
php artisan route:list           # Hiển thị tất cả route  
php artisan tinker               # Môi trường thử nghiệm code
```

4.5 Xây dựng ứng dụng web cơ bản với Laravel

Sau khi nắm vững các kiến thức về Routing, Controller, View, Eloquent ORM, Validation, Middleware và Authentication, sinh viên bắt đầu thực hành xây dựng các ứng dụng web cơ bản. Các dự án này giúp tổng hợp kiến thức đã học, rèn luyện kỹ năng làm việc nhóm, đồng thời tiếp cận quy trình phát triển ứng dụng thực tế.

4.5.1 Xây dựng website tin tức

a. Mục tiêu:

- Hiểu quy trình thiết kế và xây dựng một website quản lý tin tức (news/blog).
- Vận dụng MVC, Eloquent ORM, Blade template, Validation và Authentication.

b. Các bước thực hiện:

1. Khởi tạo dự án Laravel (Composer hoặc Laragon).
2. Tạo CSDL news_portal, xây dựng migration cho bảng posts với các trường:

- id, title, content, author_id, published_at, created_at, updated_at.
- 3. Xây dựng Model và quan hệ:
 - Post (belongsTo User).
 - User (hasMany Post).
- 4. Controller & Routing:
 - PostController: index, show, create, store, edit, update, destroy.
 - Routes trong web.php (sử dụng resource route).
- 5. Xác thực dữ liệu (Validation):
 - Bắt buộc tiêu đề, nội dung; giới hạn độ dài; published_at hợp lệ.
- 6. Giao diện (Blade Template):
 - Trang chủ: danh sách bài viết.
 - Trang chi tiết: hiển thị nội dung bài viết.
 - Trang quản trị: CRUD bài viết (chỉ dành cho người đăng nhập).
- 7. Authentication:
 - Sử dụng Breeze để tạo hệ thống đăng nhập/đăng ký.
 - Chỉ cho phép người dùng đã đăng nhập tạo/sửa/xóa bài viết.
- 8. Tính năng mở rộng:
 - Phân trang danh sách tin tức.
 - Chức năng tìm kiếm bài viết theo từ khóa.

4.5.2 Xây dựng website bán hàng (đơn giản)

a. Mục tiêu:

- Làm quen với mô hình ứng dụng thương mại điện tử nhỏ.
- Áp dụng kiến thức về cơ sở dữ liệu quan hệ, Eloquent ORM, session/cart.

b. Các bước thực hiện:

1. Tạo CSDL shop_demo, migration cho bảng:
 - products: id, name, description, price, stock, image.
 - orders: id, user_id, total_price, status.
 - order_items: id, order_id, product_id, quantity, price.
2. Model & quan hệ:
 - Product (hasMany OrderItem).
 - Order (hasMany OrderItem, belongsTo User).
 - OrderItem (belongsTo Order, belongsTo Product).
3. Chức năng chính:

- Trang chủ: hiển thị danh sách sản phẩm.
 - Chi tiết sản phẩm: hiển thị thông tin + nút “Thêm vào giỏ hàng”.
 - Giỏ hàng (Cart): dùng session để lưu sản phẩm tạm thời.
 - Đặt hàng (Checkout): yêu cầu đăng nhập, lưu đơn hàng và chi tiết vào DB.
 - Quản trị sản phẩm: CRUD sản phẩm (chỉ cho admin).
4. Validation:
- Kiểm tra số lượng tồn kho, giá sản phẩm phải hợp lệ.
 - Kiểm tra form đặt hàng.
5. Authentication & Authorization:
- Người dùng đăng nhập mới được đặt hàng.
 - Admin mới được thêm/sửa/xóa sản phẩm.
6. Tính năng mở rộng:
- Upload ảnh sản phẩm.
 - Hiển thị đơn hàng của người dùng.
 - Thêm trạng thái đơn hàng (pending, completed, cancelled).

4.6 Đóng gói và triển khai ứng dụng

Sau khi phát triển và kiểm thử thành công ứng dụng web Laravel trên môi trường cục bộ, bước cuối cùng là đóng gói và triển khai (deploy) ứng dụng lên môi trường thực tế để phục vụ người dùng. Quá trình này gồm chuẩn bị, lựa chọn môi trường triển khai và đảm bảo các yêu cầu bảo mật cơ bản.

4.6.1 Chuẩn bị ứng dụng cho triển khai

a. Cấu hình môi trường sản xuất:

- Chỉnh sửa file .env để phù hợp với server:

```
APP_ENV=production
APP_DEBUG=false
APP_KEY=base64:...
APP_URL=https://tenmiencuaban.com
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_DATABASE=ten_db
DB_USERNAME=tendangnhap
DB_PASSWORD=matkhau
```

- Tắt APP_DEBUG để tránh lộ lỗi nhạy cảm.

b. Cài đặt dependency:

```
composer install --optimize-autoloader --no-dev
npm install && npm run build
```

c. Tối ưu hóa ứng dụng:

```
php artisan config:cache  
php artisan route:cache  
php artisan view:cache
```

➔ Giúp tăng hiệu năng trên môi trường production.

d. Quản lý dữ liệu:

- Chạy migration và seeder nếu cần:

```
php artisan migrate --force
```

e. Thiết lập quyền truy cập:

- Thư mục *storage/* và *bootstrap/cache/* phải được cấp quyền ghi (write).

4.6.2 Các phương pháp triển khai

Laravel có thể triển khai trên nhiều môi trường:

- Shared Hosting:
 - Triển khai thủ công bằng cách upload mã nguồn.
 - Cấu hình Document Root trở vào thư mục public/.
 - Phù hợp với ứng dụng nhỏ, chi phí thấp.
- VPS / Cloud Server (DigitalOcean, Linode, AWS EC2...):
 - Cài đặt web server (Nginx/Apache), PHP, MySQL/PostgreSQL.
 - Dùng Laravel Forge, Envoyer hoặc tự động hóa bằng CI/CD (GitHub Actions, GitLab CI) để triển khai nhanh.
 - Phù hợp ứng dụng vừa và lớn, linh hoạt, dễ mở rộng.
- Container (Docker/Kubernetes):
 - Đóng gói ứng dụng thành container, triển khai qua Docker Compose hoặc Kubernetes.
 - Giúp dễ dàng scale, đảm bảo tính nhất quán giữa các môi trường.
 - Được khuyến nghị trong các dự án lớn, nhiều thành viên.

4.6.3 Các vấn đề bảo mật cơ bản cho ứng dụng web

1. Quản lý môi trường:
 - Không commit file .env lên Git.
 - Chỉ cho phép server đọc biến môi trường thực tế.
2. Bảo mật dữ liệu người dùng:
 - Sử dụng bcrypt hoặc argon2 để mã hóa mật khẩu.
 - Bật HTTPS để mã hóa kết nối.
3. Bảo vệ khỏi tấn công phổ biến:
 - CSRF: Laravel tích hợp sẵn CSRF token cho form.

- XSS: Blade tự động escape dữ liệu.
- SQL Injection: Eloquent ORM và Query Builder dùng Prepared Statement.
- 4. Quyền truy cập:
 - Áp dụng Authentication và Authorization hợp lý.
 - Sử dụng middleware để kiểm tra quyền hạn.
- 5. Cập nhật & giám sát:
 - Thường xuyên cập nhật framework, package.
 - Sử dụng công cụ giám sát log (Laravel Telescope, Sentry, ELK).